

RELATÓRIO DE INVESTIGAÇÃO

Estudo Comparativo de Artefactos em Memória Volátil: Infostealer vs. Ransomware

David Miranda/
Lisboa, agosto de 2026

Sumário executivo

Este relatório documenta a análise forense à memória volátil de sistemas comprometidos, evoluindo de uma investigação de incidente único para um estudo comparativo entre duas ameaças de natureza tática oposta: um infostealer (RedLine), orientado para a furtividade, e um ransomware (LockBit 4.0), desenhado para a extorsão e indisponibilização. A tese central da investigação demonstra que a pegada forense de uma ameaça não depende apenas da sua família, mas fundamentalmente da relação entre o ciclo de vida do processo e o instante exato da aquisição da memória.

O Caso A analisa o RedLine capturado em plena execução ativa, o que permitiu recuperar artefactos voláteis dinâmicos, revelando injeção de código (T1055) e tentativas de comunicação de rede. O Caso B examina o LockBit num cenário de captura póstuma, onde a ausência do processo malicioso confirma o seu comportamento de cifra-e-sai, limitando a análise ao impacto persistente, como a nota de resgate e a cifra da própria amostra na memória física.

A análise comparativa consolida o estudo, concluindo que o sucesso da investigação forense exige a adaptação da estratégia ao momento temporal do incidente e a capacidade do analista para interpretar tanto as evidências ativas como o silêncio deixado pela ausência de processos. Toda a análise obedece a uma disciplina estrita de atribuição, diferenciando os factos observados na imagem, os indícios inferidos e a capacidade intrínseca do código.

Índice

1. Introdução e objetivos	4
2. Critérios de interpretação forense	4
3. Ambiente laboratorial e ferramentas	5
3.1. Ferramentas e versões	6
4. Metodologia e cadeia de custódia	7
4.1. Integridade e cadeia de custódia	8
5. Caso A — RedLine Stealer (Infostealer)	9
5.1. Aquisição da memória	9
5.2. Dificuldades e contribuição metodológica	11
5.2.1. Falha da ferramenta de aquisição alternativa	11
5.2.2. Modo de aquisição e viabilidade de análise: PTE Remapping vs PhysicalMemory	12
5.2.3. Nota de transparência sobre os identificadores de processo	13
5.3. Comandos e plugins utilizados	15
5.4. Resultados obtidos	15
5.4.1. Sistema operativo e perfil da imagem	16
5.4.2. Processos em execução	16
5.4.3. Processos terminados ou ocultos	17
5.4.4. Relações e árvores de execução	17
5.4.5. Serviços ativos	18
5.4.6. Módulos, bibliotecas e drivers carregados	19
5.4.7. Ligações e comunicações de rede; portas abertas	19
5.4.8. Comandos executados e sessões de consola	20
5.4.9. Utilizadores e sessões existentes	21
5.4.10. Ficheiros, objetos e artefactos em memória	21
5.4.11. Indícios de injeção de código	22
5.4.12. Atribuição de família	23
5.4.13. Resumo de indicadores de comprometimento	25
5.5. Interpretação e correlação das evidências	25
5.5.1. Cronologia do incidente	25
5.5.2. Identificação e ausência de ocultação	26
5.5.3. Injeção de código	26
5.5.4. Tentativa de comunicação não concretizada	27
5.5.5. Ausência de persistência	27
5.5.6. Contexto de segurança e recolha de credenciais	27
5.5.7. Atribuição por convergência	27
5.5.8. Leitura pelo Diamond Model	27
5.5.9. Leitura pelo Malware Behavior Catalog	28
5.5.10. Quadro-síntese das correlações	28
5.5.11. Reconstrução da narrativa do incidente	29
6. Caso B — LockBit 4.0 (Ransomware)	29
6.1. Ambiente e amostra analisada	29
6.2. Detonação e efeitos observados no sistema vivo	30
6.3. Aquisição da memória e dificuldades	32
6.4. Integridade da evidência e cadeia de custódia	34
6.5. Resultados da análise em memória	36
6.6. Atribuição da família em memória	40
6.7. Quadro de indicadores de comprometimento	41
7. Análise comparativa: Furtividade contra Destruição	41
7.1. Eixo central: o momento da aquisição	42
7.2. Quadro comparativo de artefactos em memória	42
7.3. Conclusão do estudo comparativo	42
8. Classificação na Taxonomia RNCISRT	44
8.1. Classificação do Caso A (RedLine)	44
8.2. Classificação do Caso B (LockBit)	45
9. Limitações e conclusões gerais	46
9.1. Conclusões gerais	46
Referências	47

1. Introdução e objetivos

Este relatório documenta a aquisição e a análise forense da memória volátil de sistemas Windows 10 nos quais detonei, de forma controlada, duas amostras reais de malware de natureza tática oposta. O trabalho enquadra-se na unidade curricular de Análise Forense Digital e tem como objetivo demonstrar competências na captura de memória RAM e na sua análise através de ferramentas e utilitários orientados ao exame de processos, serviços, ligações de rede e demais artefactos com valor probatório. Mais do que analisar cada amostra isoladamente, o estudo compara a pegada que cada uma deixa em memória e defende uma tese central: o que o analista consegue recuperar depende tanto da família do malware como do momento em que a memória é capturada, relativamente ao ciclo de vida do processo malicioso.

Ao contrário de uma imagem de disco, cujo conteúdo é estático, a memória volátil regista o estado de execução do sistema no instante da captura. É nesse estado que residem os artefactos mais efémeros de uma infeção: regiões de código injetado, estruturas de rede, cadeias de configuração e objetos abertos pelos processos. A análise de memória é, por isso, a fonte primária de prova quando o comportamento malicioso não deixa presença duradoura em disco, como sucede com a família de malware estudada.

Para reproduzir um cenário realista, introduzi no ambiente uma amostra da família **RedLine Stealer**, obtida a partir de uma fonte pública de inteligência de ameaças. O RedLine é um infostealer, uma categoria de malware concebida para recolher e exfiltrar informação sensível do sistema comprometido. Ativo desde 2020 e distribuído sob o modelo Malware-as-a-Service, é um dos infostealers mais difundidos, capaz de recolher credenciais do Windows, informação de navegadores, carteiras de criptomoeda e dados bancários dos hospedeiros infetados, dispondo ainda da capacidade de carregar payloads de segunda fase e executar comandos recebidos do servidor de comando e controlo (Tkachenko, 2022). A escolha desta família resulta de um critério técnico: por operar segundo uma lógica de recolha rápida e por manter em memória a configuração e os artefactos de recolha, o RedLine aumenta os elementos passíveis de correlação numa análise de RAM. Como contraponto, o segundo caso analisa o LockBit 4.0, um ransomware cujo objetivo não é a recolha furtiva mas a cifra em massa e a extorsão. As duas famílias representam extremos opostos do espectro tático, furtividade contra destruição, e é essa oposição que torna a comparação da sua pegada em memória o objeto central deste estudo. O RedLine foi capturado com o processo ainda ativo; o LockBit, já depois de o seu processo ter terminado, diferença que se revela decisiva para os artefactos que cada imagem permite recuperar.

A amostra foi detonada em ambiente laboratorial isolado, para fins académicos, pelo que não se procura a atribuição a um ator de ameaça real. O foco incide na caracterização técnica do comportamento do artefacto na memória capturada, sustentada em prova documentada.

Segui, na aquisição e no exame, as orientações do NIST SP 800-86 (NIST, 2006) e da ISO/IEC 27037:2012 (ISO/IEC, 2012), registando valores de dispersão em cada etapa relevante para assegurar a integridade e a rastreabilidade da prova. A classificação do incidente segue a Taxonomia Comum da Rede Nacional de CSIRT, na versão 3.3 (RNCSIRT, 2023), apresentada na secção final.

2. Critérios de interpretação forense

Antes de apresentar as análises, estabeleço a disciplina de interpretação que aplico a ambos os casos e que sustenta todas as afirmações deste relatório. Distingo três planos, e uso deliberadamente vocabulário diferente para cada um.

Facto: o que fica provado diretamente a partir da imagem de memória, sustentado por um artefacto recuperado e por uma figura que o documenta. Uso verbos como verifiquei, identifiquei, recuperei e confirmei. Um facto não depende de conhecimento externo sobre a família de malware.

Indício: o que a evidência sugere mas não prova de forma autónoma, seja porque o artefacto é parcial, seja porque a conclusão depende de inferência. Uso formulações como os indícios apontam para, sugere ou é coerente com, e nunca as apresento como prova.

Capacidade: o que o código demonstra ser capaz de fazer, sem que a memória prove que o fez neste incidente. É o caso das cadeias de texto que revelam funcionalidades de recolha: demonstram a capacidade, não a sua execução efetiva. Mantenho esta distinção sobretudo na análise do infostealer, onde a capacidade de exfiltração não equivale à sua concretização.

Esta separação evita o erro mais comum na análise de memória: promover a comportamento observado aquilo que é apenas conhecido da literatura sobre a família. Quando um mecanismo é atribuído por comportamento documentado, e não provado pela imagem, assinalo-o explicitamente no ponto respetivo.

3. Ambiente laboratorial e ferramentas

A análise decorreu num ambiente virtualizado, montado sobre uma estação física com a VMware Workstation Pro 17. Optei pela virtualização por oferecer isolamento prático, snapshots fiáveis e uma rede interna simples de controlar, condições necessárias à detonação segura de malware. O ambiente é composto por duas máquinas virtuais, uma vítima e uma de análise, ligadas exclusivamente por uma rede host-only sem acesso ao exterior.

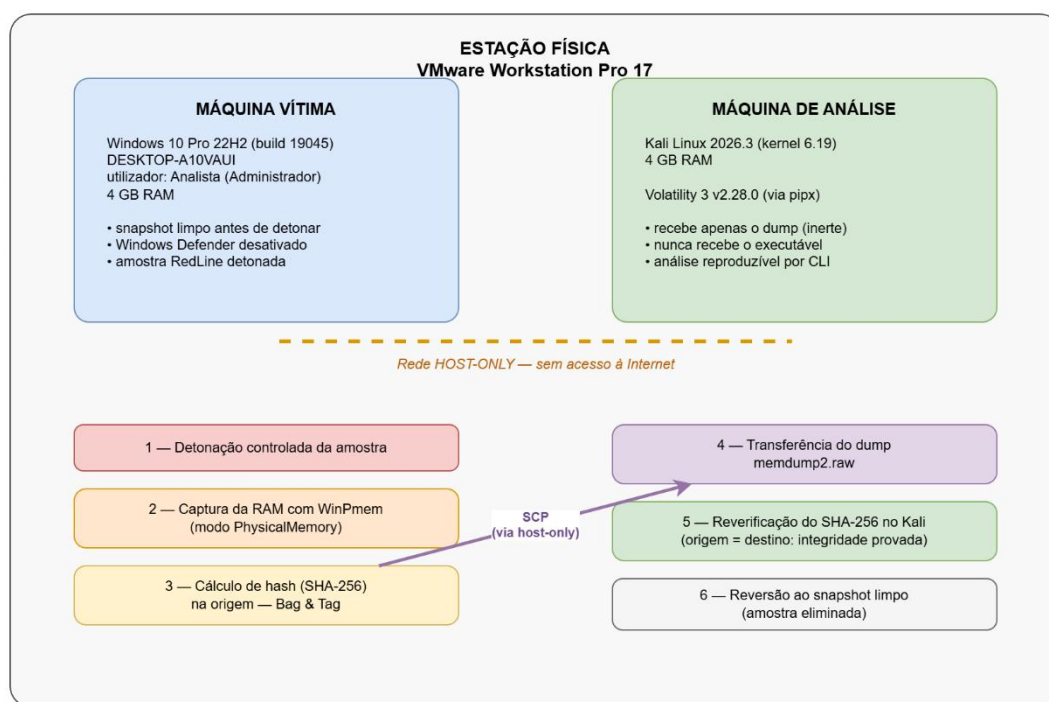


Diagrama A: Arquitetura do ambiente de laboratório e fluxo de aquisição

A máquina vítima executa o **Windows 10 Pro 22H2** (build 19045.3803), com 4 GB de RAM, sob o nome de anfitrião **DESKTOP-A10VAUI** e a conta local **Analista**, com privilégios de administrador. É nesta máquina que a amostra foi detonada e a memória capturada. A dimensão de RAM foi escolhida deliberadamente: 4 GB produzem um dump analisável sem tornar a captura e o exame desnecessariamente pesados.

A máquina de análise executa o **Kali Linux** (rolling 2026.3, kernel 6.19.14), também com 4 GB de RAM, e alberga a ferramenta de análise de memória. Esta separação entre o sistema contaminado e o sistema de

análise é uma boa prática forense: o dump, um ficheiro inerte, é o único elemento que transita da vítima para o ambiente de análise, nunca o executável malicioso.

Justifiquei a escolha do Windows 10 como sistema-alvo por três razões. A maioria das amostras de malware executa de forma fiável neste sistema; o Volatility 3 dispõe de símbolos maduros para as suas builds; e todo o material de apoio da unidade é orientado para o ambiente Windows. A rede foi mantida em host-only durante toda a detonação, garantindo que a amostra não estabelece qualquer comunicação com o exterior.

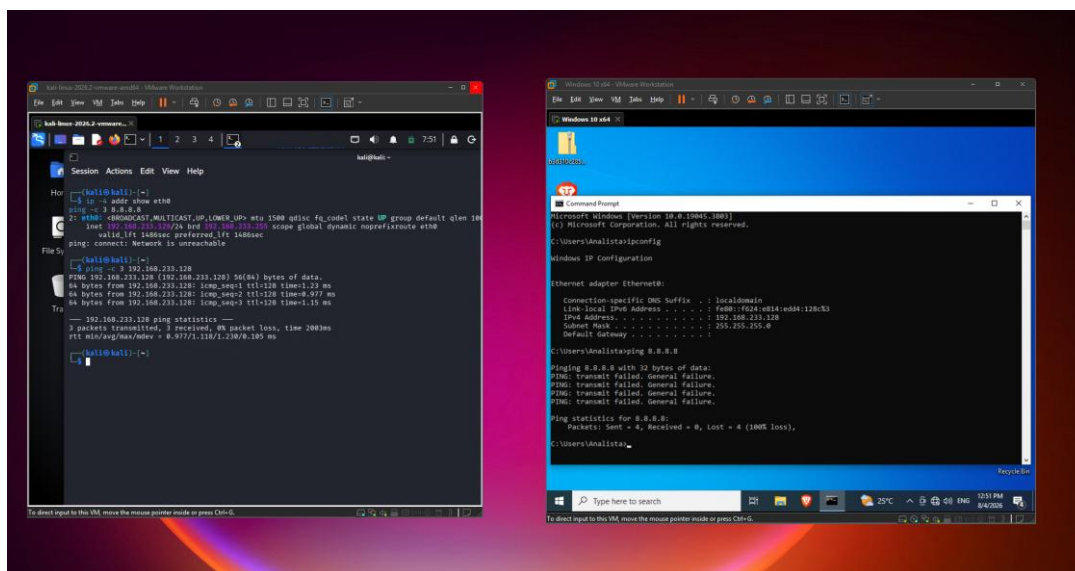


Figura 1: Confirmação do isolamento de rede: o pedido ICMP para um endereço externo falha na máquina vítima, enquanto a comunicação host-only com o Kali responde.

Confirmei o isolamento antes de qualquer detonação (Figura 1). Na máquina vítima, o pedido de eco ICMP para um endereço externo falha com perda total de pacotes, ao passo que a comunicação host-only entre a vítima e o Kali responde normalmente. O ambiente está, assim, isolado da Internet e simultaneamente funcional para a transferência controlada do dump. A detonação decorreu sempre com um snapshot limpo prévio, sem pastas partilhadas, clipboard partilhado ou arrastar-e-largar entre o anfitrião e a vítima, e a amostra foi eliminada no fim pela reversão da máquina ao snapshot limpo.

3.1. Ferramentas e versões

Reuni um conjunto de ferramentas repartido por três funções: aquisição de memória na vítima, corroboração ao vivo do estado do sistema, e análise da imagem capturada no Kali. O quadro seguinte identifica cada ferramenta, o ambiente em que corre e a respetiva versão.

Ferramenta	Ambiente	Versão	Função na investigação
WinPmem	Windows (vítima)	4.0.1	Aquisição da imagem de memória em formato RAW
FTK Imager	Windows (vítima)	8.2.0	Tentativa de aquisição alternativa (falha documentada)
Process Explorer	Windows (vítima)	Sysinternals	Corroboração ao vivo da árvore de processos e IOCs
TCPView	Windows (vítima)	Sysinternals	Corroboração ao vivo das ligações de rede
Autoruns	Windows (vítima)	Sysinternals	Verificação de mecanismos de persistência
certutil	Windows (vítima)	nativo	Cálculo de valores de dispersão na origem
Volatility 3	Kali Linux	2.28.0	Análise da imagem de memória (ferramenta principal)
sha256sum	Kali Linux	nativo	Reverificação de integridade após transferência

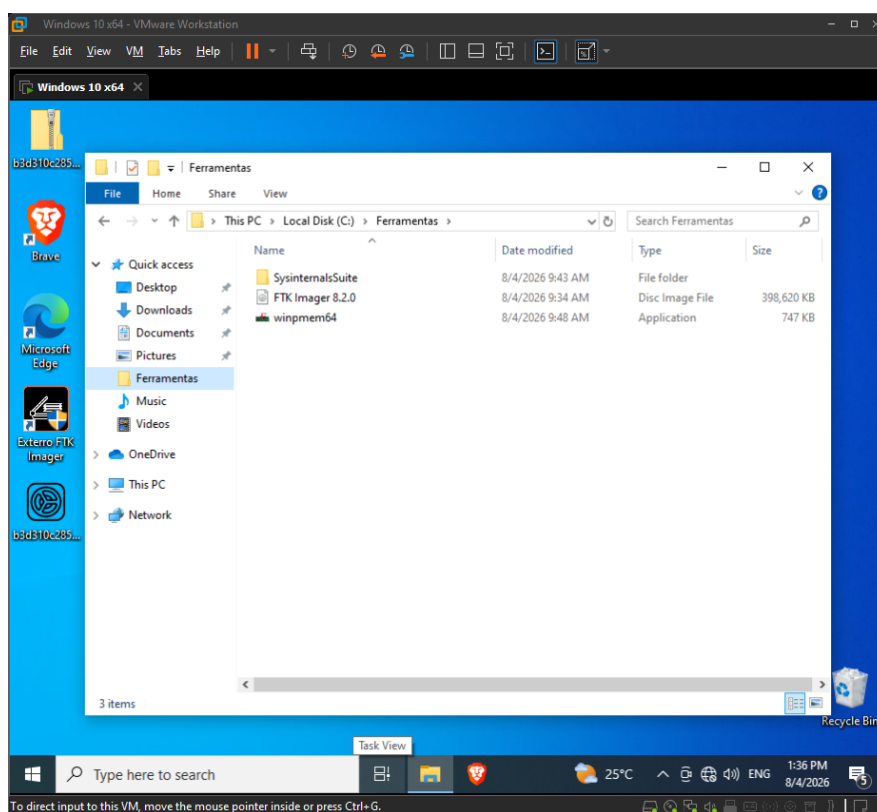


Figura 2: Inventário das ferramentas de aquisição preparadas na máquina vítima.

A ferramenta principal de análise é o **Volatility 3, versão 2.28.0**, instalado no Kali através do pipx. Registo a versão tal como o próprio ambiente a reporta, evitando confundir a numeração do framework com a da geração anterior da ferramenta. O Volatility foi complementado pelos utilitários da suite Sysinternals, executados ao vivo sobre a máquina vítima antes da captura, que servem de corroboração independente aos achados obtidos a partir do dump.

4. Metodologia e cadeia de custódia

A análise seguiu as orientações do NIST SP 800-86 (NIST, 2006) e da ISO/IEC 27037:2012 (ISO/IEC, 2012). As duas normas têm enfoques distintos e complementares. O NIST SP 800-86 põe a ênfase na integração das técnicas forenses na resposta a incidentes e organiza o processo em quatro fases: recolha, exame, análise e reporte. A ISO/IEC 27037:2012 centra-se na identificação, recolha, aquisição e preservação da prova digital, e é a norma que sustenta os requisitos de integridade e de rastreabilidade. Adotei-as em conjunto por serem complementares.

O objeto desta análise difere do de uma imagem de disco num aspeto que condiciona todo o método. Numa imagem de suporte de armazenamento, a fonte é estática e o seu conteúdo não se altera entre a aquisição e o exame. Aqui, a fonte é a memória volátil de um sistema em execução, capturada com o processo malicioso ativo. A memória não é um objeto imóvel, muda continuamente enquanto o sistema corre, e essa diferença determina a forma como conduzi a aquisição e como interpreto a integridade.

O princípio da ordem de volatilidade, estabelecido pelo NIST SP 800-86, dita que a recolha de evidências deve começar sempre pelo elemento mais volátil. Foi exatamente essa a minha abordagem. A memória é o componente mais efêmero do sistema, pelo que garanti a sua captura antes de qualquer ação destrutiva, nomeadamente antes de reverter a máquina virtual para o snapshot limpo. Como o âmbito deste trabalho se foca em exclusivo na RAM, não cheguei a percorrer toda a hierarquia descrita na norma. Apliquei o princípio, essencialmente, na decisão de priorizar a captura da memória enquanto o processo ainda estava ativo.

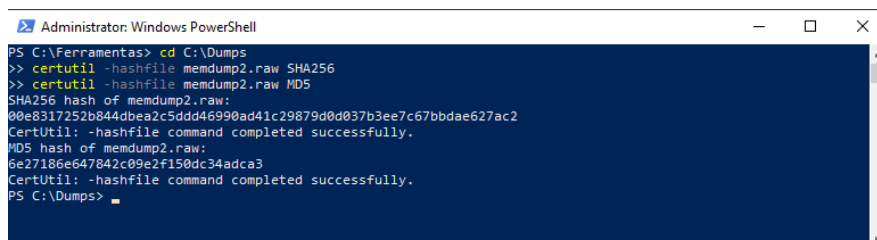
A análise apoiou-se no Volatility 3 exclusivamente por linha de comandos, abdicando de interfaces gráficas (GUI). O motivo central é a reprodutibilidade forense. O registo exato dos comandos permite a qualquer perito replicar a investigação passo a passo, algo impossível de documentar de forma auditável através de cliques numa interface. Esta escolha assegura a validade da prova e demonstra o domínio técnico das ferramentas. Para além disso, validei cada achado por mais do que uma via. Cruzei as conclusões do Volatility com os dados recolhidos ao vivo pelos utilitários Sysinternals, cumprindo assim o princípio de redundância recomendado pelo NIST SP 800-86.

4.1. Integridade e cadeia de custódia

A garantia de integridade de um dump de memória levanta uma questão que não se coloca na imagem de um disco, e que importa expor com clareza para evitar leituras erradas dos valores de dispersão apresentados.

Num suporte estático, a integridade demonstra-se por um único valor de dispersão que se mantém idêntico em toda a linha, da aquisição à análise: o mesmo objeto produz sempre a mesma impressão digital. Numa captura de memória ao vivo isso não sucede, e por uma razão que não é uma falha do procedimento. A ferramenta lê a memória física página a página enquanto o sistema continua a executar. Entre o início e o fim da leitura, processos escrevem em memória, o relógio avança e as ligações de rede evoluem. Duas capturas do mesmo sistema, feitas em instantes diferentes, correspondem a dois estados distintos de um alvo em movimento, e produzem valores de dispersão diferentes. Esta divergência é esperada e não indicia adulteração.

Daqui decorre a distinção que estrutura esta secção. A validação da integridade não procura atestar a igualdade entre capturas, mas confirmar que cada dump não foi alterado depois de capturado. Para o demonstrar, calculei o SHA-256 e o MD5 da imagem imediatamente após a aquisição, ainda no sistema de origem, com o certutil (Figura 3).



```

Administrator: Windows PowerShell
PS C:\Ferramentas> cd C:\Dumps
>> certutil -hashfile memdump2.raw SHA256
>> certutil -hashfile memdump2.raw MD5
SHA256 hash of memdump2.raw:
00e8317252b844dbea2c5ddd46990ad41c29879d0d037b3ee7c67bbdae627ac2
CertUtil: -hashfile command completed successfully.
MD5 hash of memdump2.raw:
6e27186e647842c09e2f150dc34adca3
CertUtil: -hashfile command completed successfully.
PS C:\Dumps>

```

Figura 3: Cálculo dos valores de dispersão da imagem de memória na origem (máquina vítima).

Valor de dispersão	Resultado (na origem)
SHA-256	00e8317252b844dbea2c5ddd46990ad41c29879d0d037b3ee7c67bbdae627ac2
MD5	6e27186e647842c09e2f150dc34adca3

Transferei em seguida o dump para o ambiente de análise por rede host-only, através de SCP, sem recurso a pastas partilhadas com o sistema contaminado, e recalculei o SHA-256 no Kali (Figura 4).

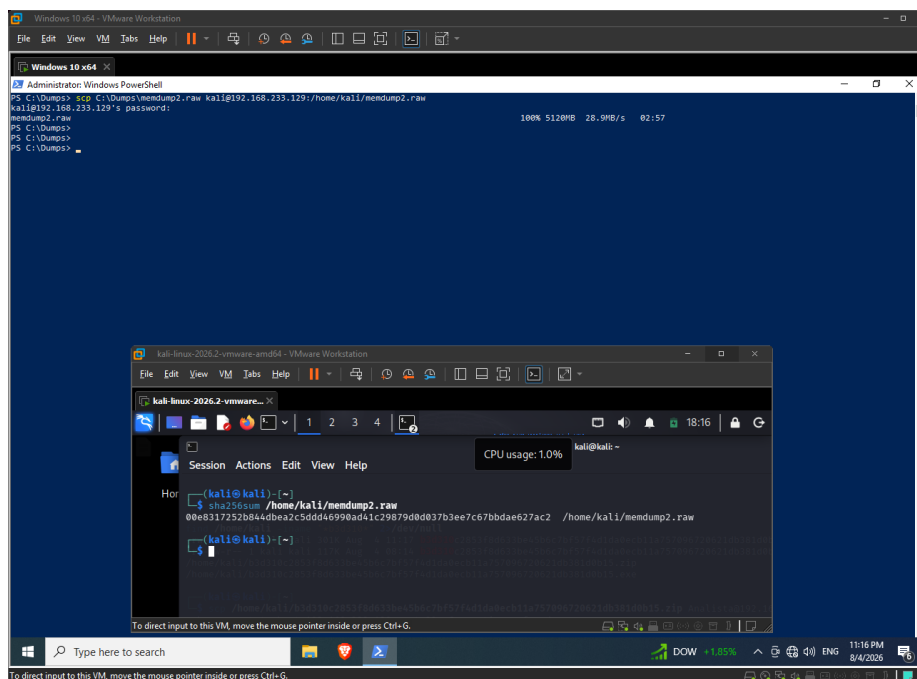


Figura 4: Transferência da imagem por SCP e reverificação do SHA-256 no ambiente de análise: o valor coincide com o da origem.

A correspondência entre o valor calculado na origem e o valor recalculado no destino demonstra que o ficheiro não sofreu alterações entre a captura e a análise. É este o elo que sustenta a cadeia de custódia da memória: ao contrário do disco, onde o mesmo valor percorre todo o processo, aqui a integridade prova-se dump a dump, comparando cada ficheiro consigo próprio antes e depois de o transferir. Registei igualmente quem capturou, quando, com que ferramenta e com que valor de dispersão, procedimento a que a sebenta da unidade curricular (Sebenta AFD, Ivo Rosa, 2025) chama Bag & Tag e que constitui o início da cadeia de custódia.

5. Caso A — RedLine Stealer (Infostealer)

5.1. Aquisição da memória

A aquisição de memória é o passo em que se decide grande parte do valor probatório da análise. Conduzi-a com a amostra ativa, num momento por mim escolhido, e antes de qualquer ação destrutiva sobre o ambiente. Antes da detonação, desliguei a proteção em tempo real do Windows Defender, condição necessária para que a amostra executasse e permanecesse em memória.

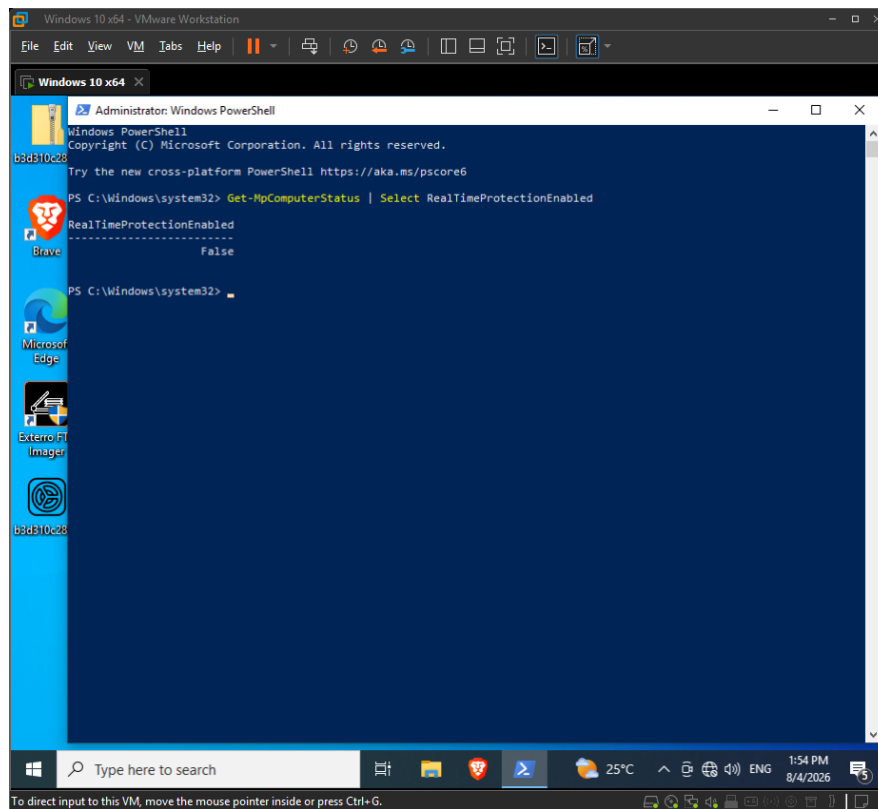


Figura 5: Verificação, antes da detonação, de que a proteção em tempo real do Windows Defender estava desativada.

Confirmei o estado do Defender por consulta direta ao seu estado de execução (Figura 5). A desativação foi deliberada e documentada: sem ela, a proteção em tempo real terminaria o processo malicioso e não restaria atividade em memória para analisar. Preparei igualmente a amostra no ambiente de trabalho da vítima, removendo a marca de segurança do Windows para permitir a execução controlada (Figura 6).

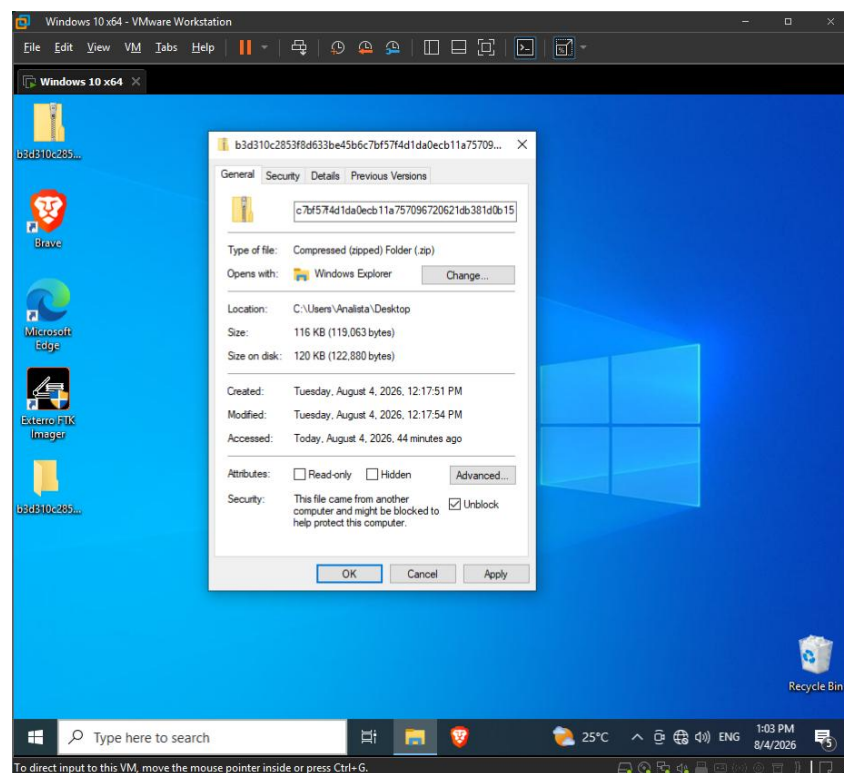


Figura 6: Propriedades do ficheiro da amostra com a marca de segurança do Windows removida, passo necessário à execução controlada.

A escolha da ferramenta de aquisição obedeceu a um critério técnico. Preferi o **WinPmem** por gerar uma imagem RAW da memória física, formato que preserva as estruturas de rede em memória e assegura que os plugins que dependem delas funcionam. Evitei deliberadamente formatos de crash dump, cuja limitação conhecida é não preservar essas estruturas. A ferramenta foi executada por linha de comandos, com privilégios de administrador, com a amostra ativa:

```
cd C:\Ferramentas\
.\winpmem64.exe C:\Dumps\memdump2.raw
```

A imagem resultante tem 5 368 709 120 bytes (5 GB), valor que corresponde à totalidade do espaço de memória física endereçável da máquina. Introduzi e detonei a amostra manualmente, diretamente no ambiente de trabalho do utilizador. Como se trata de um método de introdução controlada em laboratório, e não do vetor de infeção real da família, a genealogia de processos observada deve ser lida apenas como contexto do exercício. Não constitui uma técnica autónoma do malware. A secção seguinte foca-se na garantia de integridade desta evidência. Adicionalmente, na secção de dificuldades, documento um obstáculo relevante relacionado com o modo de aquisição do WinPmem e a forma como o ultrapassei.

5.2. Dificuldades e contribuição metodológica

Durante a aquisição enfrentei duas dificuldades relevantes, que documento por terem valor metodológico e por terem condicionado o procedimento. A primeira respeita à ferramenta de aquisição alternativa; a segunda, mais importante, respeita ao modo de captura do WinPmem e à viabilidade de análise do resultado.

5.2.1. Falha da ferramenta de aquisição alternativa

Tentei, em primeiro lugar, a aquisição com o FTK Imager. A ferramenta falhou de forma persistente com o erro Could not start driver, mesmo após ativar o modo de teste de assinaturas do Windows (Figura 7). O WinPmem capturou com sucesso no mesmo ambiente, após a ativação do test signing, o que demonstra que a causa da falha do FTK não é a assinatura de drivers, mas uma incompatibilidade própria do seu driver de captura com esta build. A falha do FTK é, ainda assim, um achado legítimo de metodologia, e não uma lacuna: a tentativa de aquisição por ferramentas independentes é uma boa prática forense.

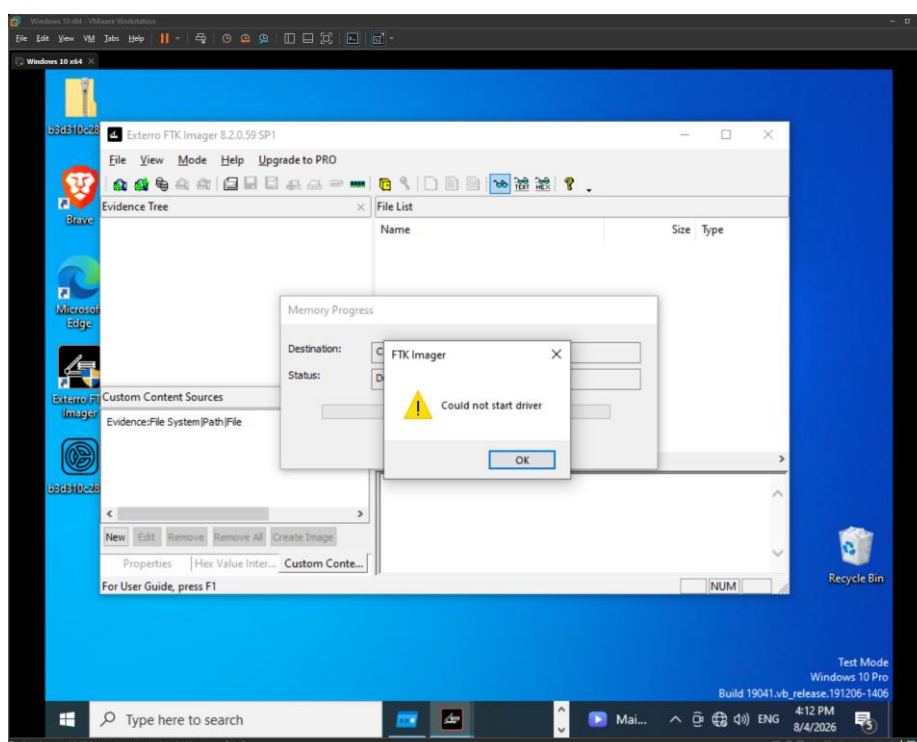
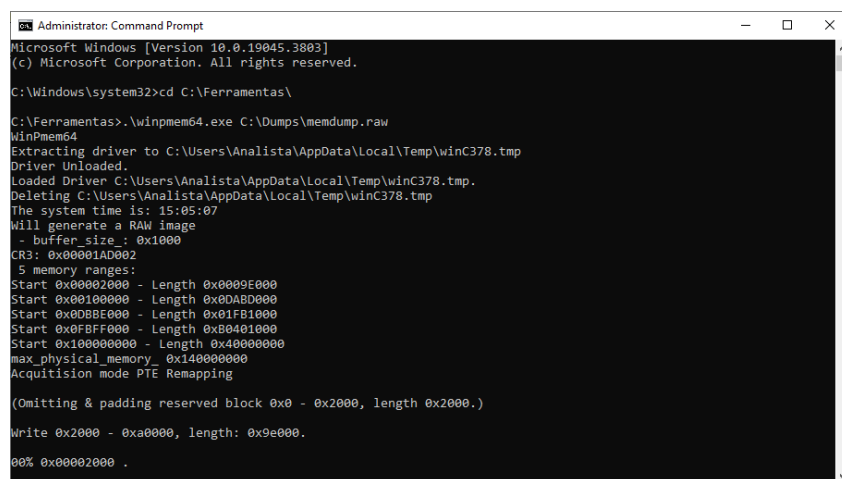


Figura 7: O FTK Imager falha a aquisição com o erro Could not start driver, mesmo com o modo de teste ativo.

5.2.2. Modo de aquisição e viabilidade de análise: PTE Remapping vs PhysicalMemory

A dificuldade central deste trabalho revelou-se na primeira aquisição bem-sucedida com o WinPmem. A ferramenta, na predefinição para sistemas de 64 bits, usa o modo PTE Remapping (Figura 8). A imagem daí resultante, embora concluída sem erros, revelou-se não analisável pelo Volatility 3. O diagnóstico foi conclusivo: o windows.info reportava a ausência do requisito de símbolos do kernel, e a pesquisa de banners devolvia uma tabela vazia, ainda que a imagem contivesse conteúdo (exportações do ntoskrnl, cadeias de texto do sistema).



```
Administrator: Command Prompt
Microsoft Windows [Version 10.0.19045.3803]
(c) Microsoft Corporation. All rights reserved.

C:\Windows\system32>cd C:\Ferramentas\

C:\Ferramentas>.winpmem64.exe C:\Dumps\memdump.raw
WinPmem64
Extracting driver to C:\Users\Analista\AppData\Local\Temp\winC378.tmp
Driver Unloaded.
Loaded Driver C:\Users\Analista\AppData\Local\Temp\winC378.tmp.
Deleting C:\Users\Analista\AppData\Local\Temp\winC378.tmp
The system time is: 15:05:07
Will generate a RAW image
-- buffer_size: 0x1000
CR3: 0x00001A002
5 memory ranges:
Start 0x00002000 - Length 0x0009E000
Start 0x00100000 - Length 0x00A8D000
Start 0x00B8E000 - Length 0x01FB1000
Start 0x0FBFF000 - Length 0x00401000
Start 0x100000000 - Length 0x40000000
max_physical_memory_ 0x140000000
Acquisition mode PTE Remapping

(Omitting & padding reserved block 0x0 - 0x2000, length 0x2000.)

Write 0x2000 - 0xa000, length: 0x9e000.
00% 0x00002000 .
```

Figura 8: Primeira aquisição com WinPmem em modo PTE Remapping, cuja imagem resultou não analisável pelo Volatility 3.

O problema ocorre porque o modo PTE Remapping reorganiza as páginas de memória: a assinatura do kernel (o par `ntkrnlmp.pdb` e o respetivo GUID) deixa de ficar no deslocamento físico esperado, e o mecanismo de correspondência de símbolos do Volatility 3 falha. A imagem tem conteúdo, mas o framework não consegue localizar o ponto de ancoragem de que precisa para reconstruir as estruturas do sistema.

A correção consistiu em repetir a aquisição forçando o modo PhysicalMemory (Figura 9). Esta imagem RAW linear preservou a assinatura do kernel no deslocamento esperado, o que permitiu ao Volatility 3 casar os símbolos e tornar a imagem plenamente analisável. É esta a segunda imagem, e é sobre ela que toda a análise incide.

```

Administrator: Windows PowerShell
Deleting C:\Users\Analista\AppData\Local\Temp\win1837.tmp
The system time is: 19:38:08
Will generate a RAW image
- buffer size: 0x1000
CR3: 0x00001AD002
5 memory ranges:
Start 0x00002000 - Length 0x0009E000
Start 0x00100000 - Length 0x00DABD000
Start 0x00BBE000 - Length 0x01F81000
Start 0x0FBFF000 - Length 0x00401000
Start 0x10000000 - Length 0x40000000
max_physical_memory_ 0x140000000
Acquisition mode \\.\PhysicalMemory

(Omitting & padding reserved block 0x0 - 0x2000, length 0x2000.)
Write 0x2000 - 0xa000, length: 0x9e000.
30% 0x00002000 ..

(Omitting & padding reserved block 0xA0000 - 0x100000, length 0x60000.)
Write 0x100000 - 0xdbbd000, length: 0xdabd000.
30% 0x00100000 .....

(Omitting & padding reserved block 0xDBD000 - 0xDBBE000, length 0x1000.)
Write 0xdbbe000 - 0xfb6f000, length: 0x1fb1000.
34% 0x00BBE000 ..

(Omitting & padding reserved block 0xFB6F000 - 0xFBFF000, length 0x90000.)
Write 0xfbff000 - 0xc0000000, length: 0xb0401000.
34% 0x0FBFF000 .....
36% 0x41BFF000 .....
36% 0x73BFF000 .....
31% 0xA5BFF000 .....

(Omitting & padding reserved block 0xC0000000 - 0x100000000, length 0x40000000.)
Write 0x100000000 - 0x140000000, length: 0x40000000.
30% 0x100000000 .....
35% 0x132000000 .....
The system time is: 19:38:20
Driver Unloaded.
PS C:\Ferramentas>

```

Figura 9: Segunda aquisição forçando o modo PhysicalMemory: a imagem linear preservou a assinatura do kernel e tornou-se analisável.

Retiro daqui uma recomendação prática: em sistemas de 64 bits, quando o alvo do exame é o Volatility 3, deve forçar-se o modo de aquisição PhysicalMemory em detrimento da predefinição PTE Remapping. O primeiro produz uma imagem RAW linear cujos símbolos do kernel o Volatility localiza; o segundo reorganiza as páginas de forma incompatível com esse mecanismo. Esta observação converte um problema de aquisição numa contribuição metodológica reaproveitável.

5.2.3. Nota de transparência sobre os identificadores de processo

Durante a preparação realizei detonações sucessivas da amostra. A análise ao vivo com os utilitários Sysinternals documentou uma instância anterior, identificada por um identificador de processo distinto (Figura 10). A aquisição de memória exigiu o reinício do sistema para ativar o modo de teste de assinaturas necessário ao WinPmem. Esse reinício originou nova detonação do mesmo binário, atribuída ao PID 1040, que constitui o objeto do dump analisado. Confirmei tratar-se do mesmo executável pela coincidência dos indicadores (versão interna, carimbo de compilação ausente, arquitetura de 32 bits), diferindo apenas o identificador de processo por corresponderem a execuções distintas.

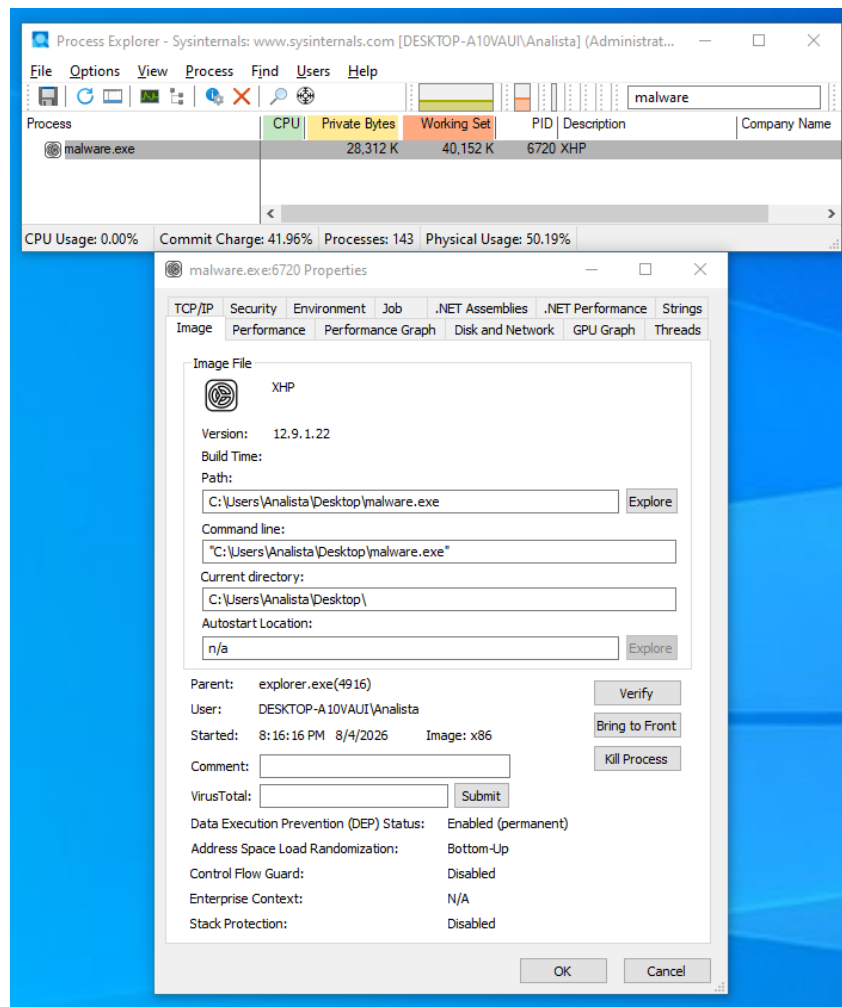


Figura 10: Process Explorer com a amostra em execução ao vivo: indicadores do binário (nome interno XHP, carimbo de compilação vazio, arquitetura x86, mitigações desativadas).

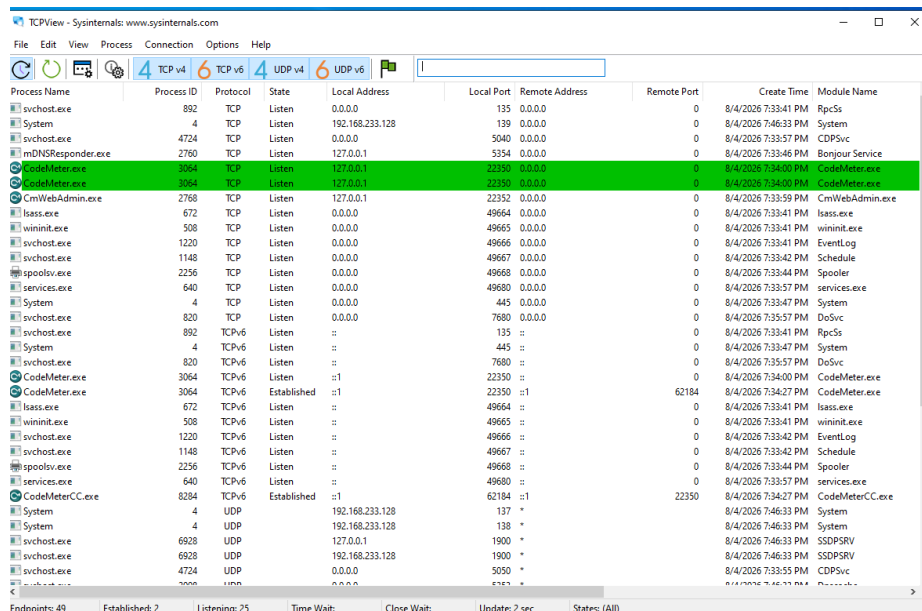


Figura 11: TCPView ao vivo: ausência de ligações de rede externas estabelecidas pela amostra, coerente com o isolamento.

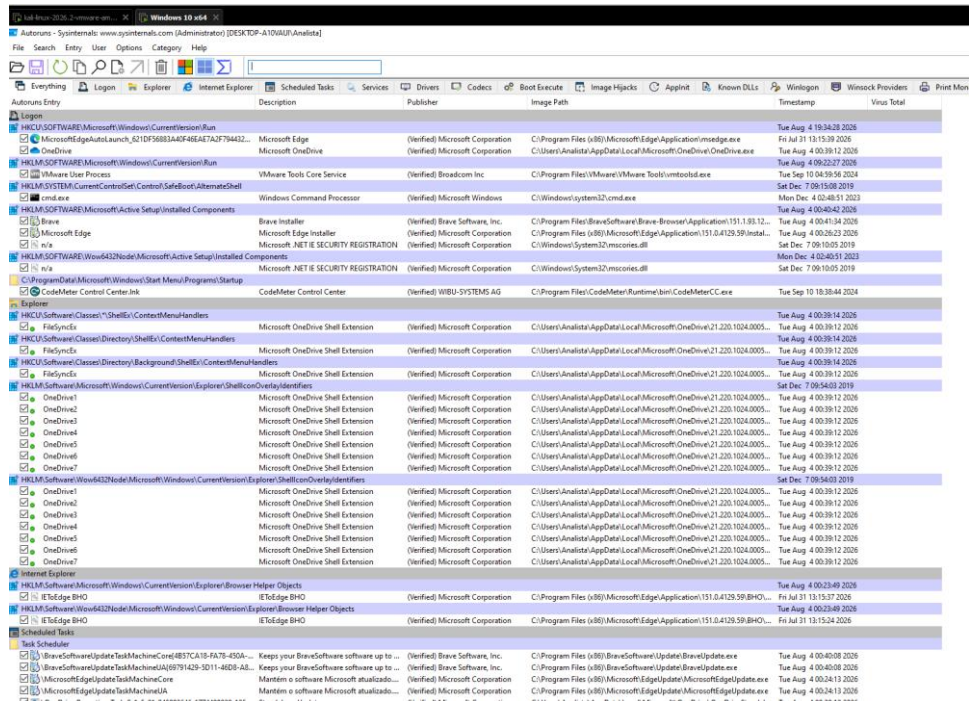


Figura 12: Autoruns após a detonação: nenhuma entrada de arranque automático criada pela amostra.

5.3. Comandos e plugins utilizados

O quadro seguinte associa cada plugin do Volatility 3 ao elemento mínimo da análise que ele responde, conforme os pontos enunciados para o exercício. A invocação-base foi `vol -f memdump2.raw <plugin>`, ficando cada resultado documentado nas figuras desta secção.

Plugin	Elemento da análise
windows.info	Sistema operativo e perfil da imagem
windows.pslist	Processos em execução
windows.psscan	Processos terminados ou ocultos
windows.pstree	Relações e árvores de execução
windows.svcscan	Serviços ativos
windows.dlllist	Módulos, bibliotecas e drivers carregados
windows.netscan	Ligações de rede e portas abertas com processo associado
windows.cmdline / windows.consoles	Comandos executados e sessões de consola
windows.sessions / windows.getsids	Utilizadores e sessões existentes
windows.handles / windows.dumpfiles	Ficheiros, objetos e artefactos em memória
windows.malfind	Injeção de código e indícios de atividade anómala

A ordem de execução seguiu a lógica da investigação: primeiro a caracterização do sistema e a identificação do processo suspeito, depois o exame do seu comportamento (rede, serviços, bibliotecas, sessões, objetos) e, por fim, a deteção de injeção de código e a extração do binário para atribuição de família.

5.4. Resultados obtidos

Apresento os resultados pela ordem dos elementos mínimos da análise. Cada figura é chamada pelo texto que a sustenta; a interpretação técnica de cada achado, e a sua correlação, é aprofundada na secção 9.

5.4.1. Sistema operativo e perfil da imagem

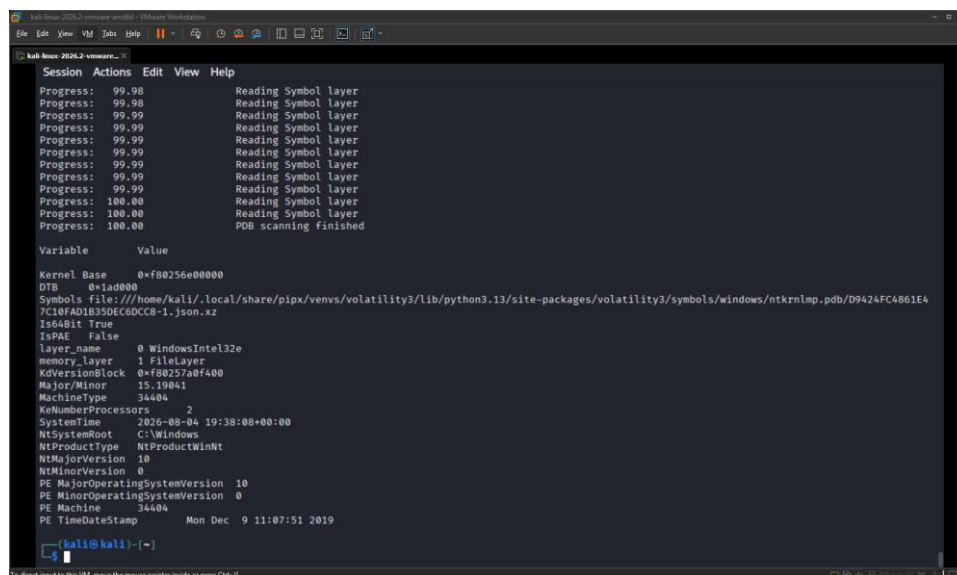


Figura 13: windows.info: casamento dos símbolos do kernel e perfil do sistema operativo da imagem de memória.

O plugin windows.info confirmou a viabilidade de análise da imagem (Figura 13). Os símbolos do kernel casaram (ntkrnlmp.pdb, GUID D9424FC4861E47C10FAD1B35DEC6DCC8), o sistema foi identificado como Windows 10 de 64 bits (major/minor 15.19041, dois processadores) e a data do sistema no dump, 2026-08-04 19:38:08 UTC, corresponde ao momento da captura. O casamento dos símbolos é a condição que valida todo o exame subsequente.

5.4.2. Processos em execução

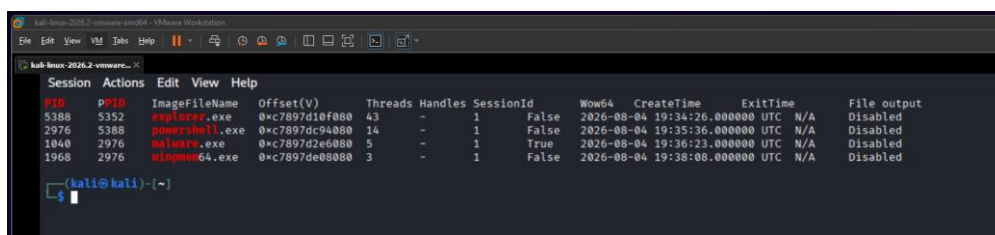


Figura 14: windows.pslist: o processo malware.exe (PID 1040) e a cadeia de execução.

O windows.pslist identificou o processo suspeito malware.exe, PID 1040, a executar com Wow64 a True (arquitetura de 32 bits) e hora de criação 2026-08-04 19:36:23 UTC (Figura 14). A cadeia explorer.exe (5388), powershell.exe (2976), malware.exe (1040) é visível na listagem. Presente está também o processo winpmem64.exe, correspondente à própria ferramenta de aquisição, cuja existência no dump é normal e esperada.

5.4.3. Processos terminados ou ocultos

```

(kali@kali)~$ vol -f /home/kali/memdump2.raw windows.psscan > out_psscan.txt

(kali@kali)~$ grep -i malware out_psscan.txt
1040 2976 malware.exe 0xc7897d2e6080 5 - 1 True 2026-08-04 19:36:23.000000 UTC N/A Disabled

(kali@kali)~$ grep -c "" out_pslist.txt
134
141

(kali@kali)~$ comm -13 <(awk '{print $1}' out_pslist.txt | sort -u) <(awk '{print $1}' out_psscan.txt | sort -u)
4840
7356
7728

(kali@kali)~$ grep -E "(4840|7356|7728)\s" out_psscan.txt
4840 1212 taskhostw.exe 0xc7897da14080 0 - 0 False 2026-08-04 19:36:51.000000 UTC 2026-08-04 19:36:51.000000 UTC D
7356 588 svchost.exe 0xc7897dd77080 0 - 0 False 2026-08-04 19:35:59.000000 UTC 2026-08-04 19:37:00.000000 UTC D
7728 776 WmiPrvSE.exe 0xc7897df84080 0 - 0 False 2026-08-04 19:35:43.000000 UTC 2026-08-04 19:37:28.000000 UTC D
  
```

Figura 15: Comparação windows.pslist vs windows.psscan: o processo suspeito surge em ambos com o mesmo deslocamento, sem ocultação.

Comparei o windows.pslist com o windows.psscan, que varre as estruturas de memória e recupera processos desassociados da lista ativa (Figura 15). O processo suspeito surge em ambos, com o mesmo deslocamento físico, pelo que não está oculto. Os três processos exclusivos do psscan (PID 4840, 7356, 7728) apresentam zero threads e ExitTime preenchido, correspondendo a processos terminados residuais e benignos. Não há indício de ocultação por desassociação da lista de processos.

5.4.4. Relações e árvores de execução

```

(kali@kali)~$ vol -f /home/kali/memdump2.raw windows.pstree > out_pstree.txt

(kali@kali)~$ grep -iE "malware|powershell|explorer" out_pstree.txt
***** 5388 explorer.exe 0xc7897d10f080 43 - 1 False 2026-08-04 19:34:26.000000 UTC N/A \Device\HarddiskV
o\ume3\Windows\explorer.exe C:\Windows\Explorer.EXE C:\Windows\Explorer.EXE
***** 2976 powershell.exe 0xc7897dc94080 14 - 1 False 2026-08-04 19:35:36.000000 UTC N/A \Device\HarddiskV
o\ume3\Windows\System32\WindowsPowerShell\v1.0\powershell.exe "C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe"
***** 1040 powershell.exe 0xc7897d2e6080 5 - 1 True 2026-08-04 19:36:23.000000 UTC N/A \Device\HarddiskV
o\ume3\Users\Analista\Desktop\malware.exe "C:\Users\Analista\Desktop\malware.exe" C:\Users\Analista\Desktop\malware.exe
  
```

Figura 16: windows.pstree: genealogia explorer.exe → powershell.exe → malware.exe.

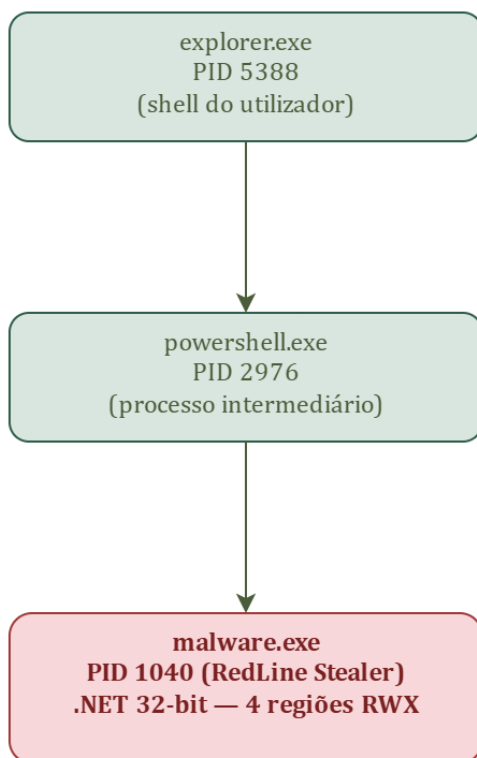


Figura 17: Árvore de processos: *genealogia explorer.exe (PID 5388), powershell.exe (PID 2976) e malware.exe (PID 1040)*.

O windows.pstree confirmou a genealogia de execução (Figura 16): explorer.exe lançou powershell.exe, que por sua vez lançou malware.exe, com a linha de comando completa a apontar para o ambiente de trabalho do utilizador. Como referido, o processo pai powershell.exe corresponde ao método de detonação manual que utilizei no laboratório, e não a uma técnica de ataque; fica registado para transparência.

5.4.5. Serviços ativos

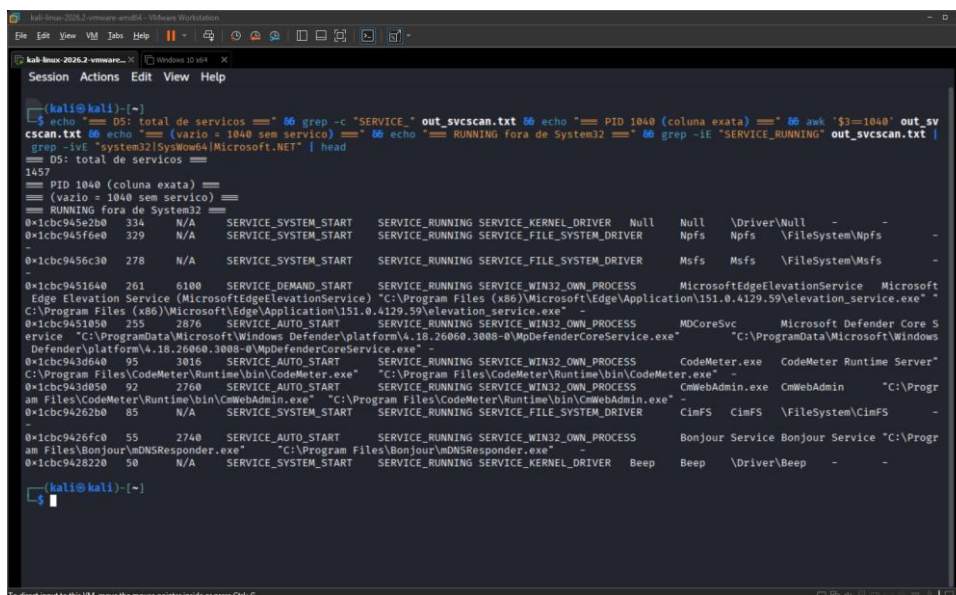
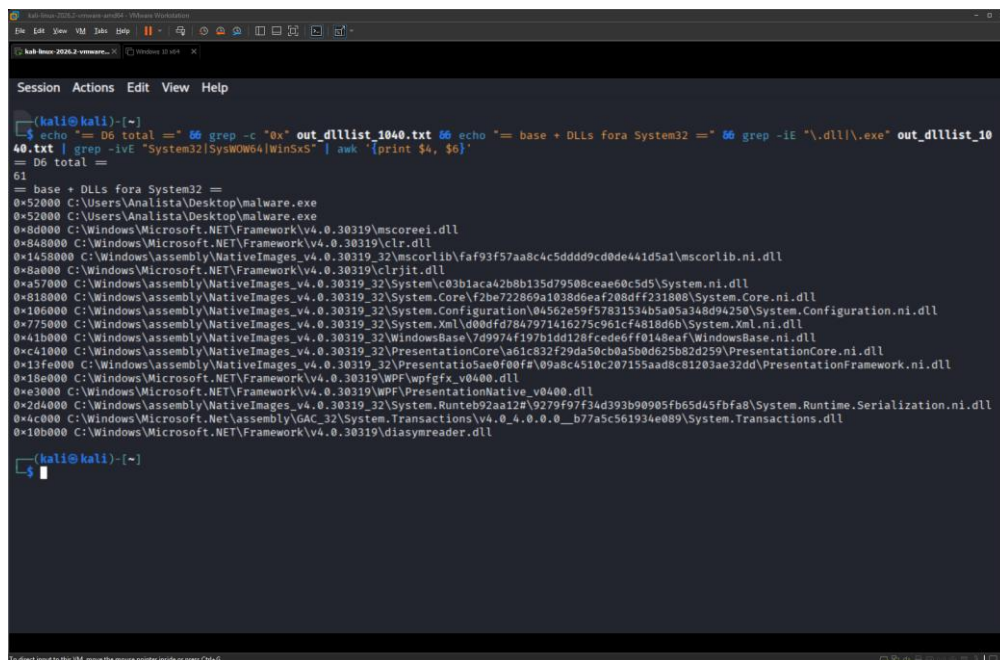


Figura 18: windows.svcscan: nenhum dos servicos enumerados está associado ao processo suspeito.

O windows.svcscan enumerou 1457 serviços (Figura 18). Filtrando pela coluna do identificador de processo, nenhum está associado ao PID 1040: a amostra não criou qualquer serviço. Os serviços em execução fora da pasta de sistema correspondem a software legítimo de terceiros. Este é um achado

negativo com valor probatório, pois exclui a persistência por serviço, comportamento coerente com a família estudada.

5.4.6. Módulos, bibliotecas e drivers carregados



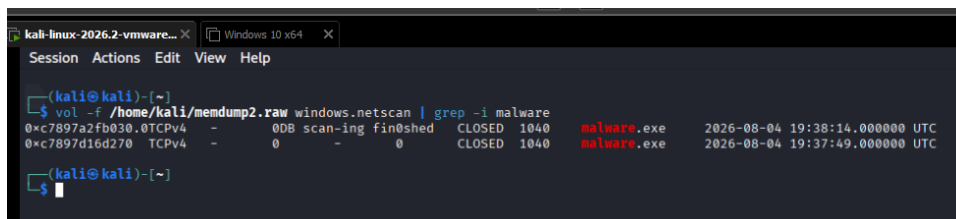
```

(kali@kali)~$ out_dlllist 1040.txt
61
base + DLLs fora System32 =
0x52000 C:\Users\Analista\Desktop\malware.exe
0x52000 C:\Users\Analista\Desktop\malware.exe
0x8d000 C:\Windows\Microsoft.NET\Framework\v4.0.30319\mscorlib.dll
0x8d000 C:\Windows\Microsoft.NET\Framework\v4.0.30319\clr.dll
0x145000 C:\Windows\assembly\NativeImages_v4.0.30319_32\mscorlib\af93f57aa8c4c5ddd9cd0de441d5a1\mscorlib.ni.dll
0x8a000 C:\Windows\Microsoft.NET\Framework\v4.0.30319\clrjit.dll
0x57000 C:\Windows\assembly\NativeImages_v4.0.30319_32\System\c03blaca42b8b135d79508ceae0c5d5\System.ni.dll
0x818000 C:\Windows\assembly\NativeImages_v4.0.30319_32\System.Core\fb2e722869a1038d6eaf208df231808\System.Core.ni.dll
0x106000 C:\Windows\assembly\NativeImages_v4.0.30319_32\System.Configuration\04562e59f57831534b5a05a348d94250\System.Configuration.ni.dll
0x775000 C:\Windows\assembly\NativeImages_v4.0.30319_32\System.Xml\d00df7847971416275c961cf4818d6b\System.Xml.ni.dll
0x419000 C:\Windows\assembly\NativeImages_v4.0.30319_32\WindowsBase\7d9974f197b1d128fced6f8148ea\WindowsBase.ni.dll
0x419000 C:\Windows\assembly\NativeImages_v4.0.30319_32\PresentationCore\0a61c827f29da50c8ba5b0d625b2d259\PresentationCore.ni.dll
0x13fe000 C:\Windows\assembly\NativeImages_v4.0.30319_32\PresentationFramework\09a8c4510c207155aad8c81203ae32dd\PresentationFramework.ni.dll
0x18e000 C:\Windows\Microsoft.NET\Framework\v4.0.30319\WPFWpf\wpfgfx_v0400.dll
0x3000 C:\Windows\Microsoft.NET\Framework\v4.0.30319\WPFWpf\PresentationNative_v0400.dll
0x2d4000 C:\Windows\assembly\NativeImages_v4.0.30319_32\System.Runtime\9279f97f34d393b90905fb65d45fba8\System.Runtime.Serialization.ni.dll
0x4c000 C:\Windows\Microsoft.NET\assembly\GAC_32\System.Transactions\v4.0.0.0__b77a5c561934e089\System.Transactions.dll
0x10b000 C:\Windows\Microsoft.NET\Framework\v4.0.30319\diasymreader.dll
  
```

Figura 19: windows.dlllist do PID 1040: carregamento do Common Language Runtime do .NET Framework.

O windows.dlllist sobre o PID 1040 listou 61 bibliotecas (Figura 19). O executável reside no ambiente de trabalho e carrega o Common Language Runtime do .NET Framework (clr.dll, mscorlib.dll, clrjit.dll), confirmando tratar-se de um binário .NET (managed). Não se observou carregamento lateral de bibliotecas anômalas. Este carregamento do runtime .NET é coerente com a supressão do carimbo de compilação do executável observada na análise ao vivo (Figura 10), um binário .NET com metadados de compilação ausentes.

5.4.7. Ligações e comunicações de rede; portas abertas



```

(kali@kali)~$ windows.netscan | grep -i malware
0xc7897a2fb030.0TCPv4 - 00B scan-ing fin0shed CLOSED 1040 malware.exe 2026-08-04 19:38:14.000000 UTC
0xc7897d16d270 TCPv4 - 0 - 0 CLOSED 1040 malware.exe 2026-08-04 19:37:49.000000 UTC
  
```

Figura 20: windows.netscan: dois sockets TCPv4 associados ao PID 1040, ambos no estado CLOSED.

O windows.netscan identificou dois sockets TCPv4 associados ao PID 1040, ambos no estado CLOSED, com carimbos temporais 19:37:49 e 19:38:14 UTC, posteriores à detonação e coerentes com ela (Figura 20). O endereço de destino não é recuperável, por os sockets estarem fechados e as respectivas estruturas parcialmente sobrescritas. Houve, portanto, uma tentativa de comunicação de rede, indício de contacto com infraestrutura de comando e controlo, mas o destino não é determinável a partir deste dump, resultado coerente com o ambiente isolado.

5.4.8. Comandos executados e sessões de console

```

kali-linux-2026.2-vmware-amd64 - VMware Workstation
File Edit View VM Tabs Help
kali-linux-2026.2-vmware... X Windows 10 x64 X
Session Actions Edit View Help
(kali@kali)-[~]
$ vol -f /home/kali/memdump2.raw windows.cmdline --pid 1040 2976
Volatility 3 Framework 2.28.0
Progress: 100.00 PDB scanning finished
PID Process Args
2976 powershell.exe "C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe"
1040 malware.exe "C:\Users\Analista\Desktop\malware.exe"
(kali@kali)-[~]
$

```

Figura 21: windows.cmdline: linha de comando do processo suspeito, sem argumentos adicionais.

O windows.cmdline mostrou que o processo suspeito foi executado diretamente a partir de C:\Users\Analista\Desktop\malware.exe, sem argumentos adicionais (Figura 21). A ausência de argumentos na linha de comando não exclui atividade, uma vez que um binário .NET desta natureza não depende deles para operar.

```

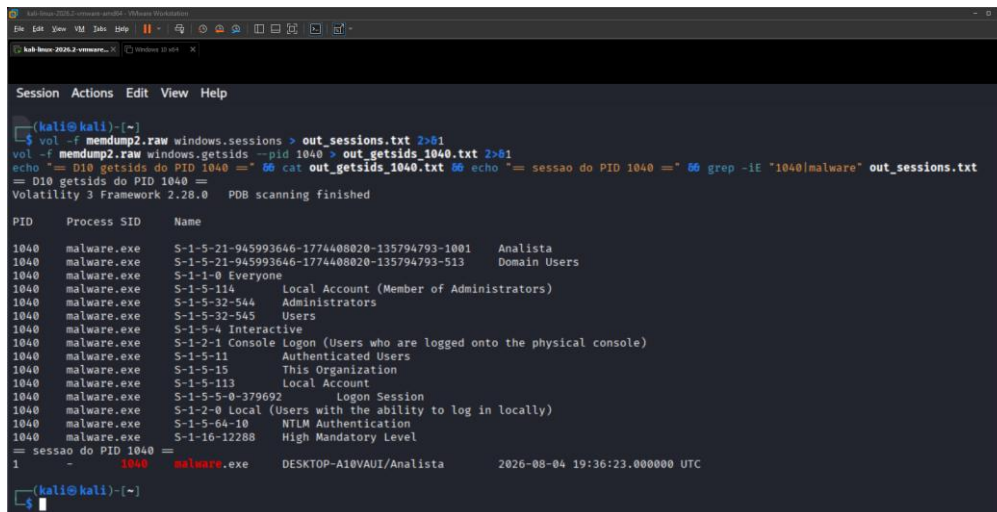
kali-linux-2026.2-vmware-amd64 - VMware Workstation
File Edit View VM Tabs Help
kali-linux-2026.2-vmware... X Windows 10 x64 X
Session Actions Edit View Help
(kali@kali)-[~]
$ vol -f memdump2.raw windows.consoles
Volatility 3 Framework 2.28.0
Progress: 100.00 PDB scanning finished
PID Process ConsoleInfo Property Address Data
3052 conhost.exe 0x7ff79b082e68 _CONSOLE_INFORMATION N/A Console Information Not Found
(kali@kali)-[~]
$ vol -f memdump2.raw windows.cmdscan
Volatility 3 Framework 2.28.0
Progress: 100.00 PDB scanning finished
PID Process ConsoleInfo Property Address Data
3052 conhost.exe 0x1a3a98586d0 _COMMAND_HISTORY 0x1a3a98586d0 None
* 3052 conhost.exe 0x1a3a98586d0 _COMMAND_HISTORY.Application 0x1a3a9858700 winpmem64.exe
* 3052 conhost.exe 0x1a3a98586d0 _COMMAND_HISTORY.ProcessHandle 0x1a3a9c2f520 0x35c
* 3052 conhost.exe 0x1a3a98586d0 _COMMAND_HISTORY.CommandCount N/A 0
* 3052 conhost.exe 0x1a3a98586d0 _COMMAND_HISTORY.LastDisplayed 0x1a3a985872c -1
* 3052 conhost.exe 0x1a3a98586d0 _COMMAND_HISTORY.CommandCountMax 0x1a3a98586f8 50
* 3052 conhost.exe 0x1a3a98586d0 _COMMAND_HISTORY.CommandBucket 0x1a3a98586e0
3052 conhost.exe 0x1a3a98592a0 _COMMAND_HISTORY 0x1a3a98592a0 None
* 3052 conhost.exe 0x1a3a98592a0 _COMMAND_HISTORY.Application 0x1a3a98592d0 powershell.exe
* 3052 conhost.exe 0x1a3a98592a0 _COMMAND_HISTORY.ProcessHandle 0x1a3a7d96720 0xf8
* 3052 conhost.exe 0x1a3a98592a0 _COMMAND_HISTORY.CommandCount N/A 0
* 3052 conhost.exe 0x1a3a98592a0 _COMMAND_HISTORY.LastDisplayed 0x1a3a98592fc -1
* 3052 conhost.exe 0x1a3a98592a0 _COMMAND_HISTORY.CommandCountMax 0x1a3a98592c8 50
* 3052 conhost.exe 0x1a3a98592a0 _COMMAND_HISTORY.CommandBucket 0x1a3a98592b0
(kali@kali)-[~]
$

```

Figura 22: windows.consoles e windows.cmdscan: buffers de console associados às ferramentas do analista, não à amostra.

O windows.consoles e o windows.cmdscan não recuperaram qualquer comando da amostra (Figura 22). Os únicos buffers de console presentes pertencem às minhas ferramentas de aquisição, o winpmem64.exe e o powershell.exe, e estavam vazios. O resultado é coerente com o RedLine ser uma aplicação .NET sem console própria.

5.4.9. Utilizadores e sessões existentes



```

(kali@kali)-[~]
└─$ vol -f memdump2.raw windows.sessions > out_sessions.txt 2>&1
vol -f memdump2.raw windows.getsids --pid 1040 > out_getsids_1040.txt 2>&1
echo "D10 getsids do PID 1040 =" 66 cat out_getsids_1040.txt 66 echo " = sessão do PID 1040 =" 66 grep -iE "1040|malware" out_sessions.txt
= D10 getsids do PID 1040 =
Volatility 3 Framework 2.20.0 PDB scanning finished

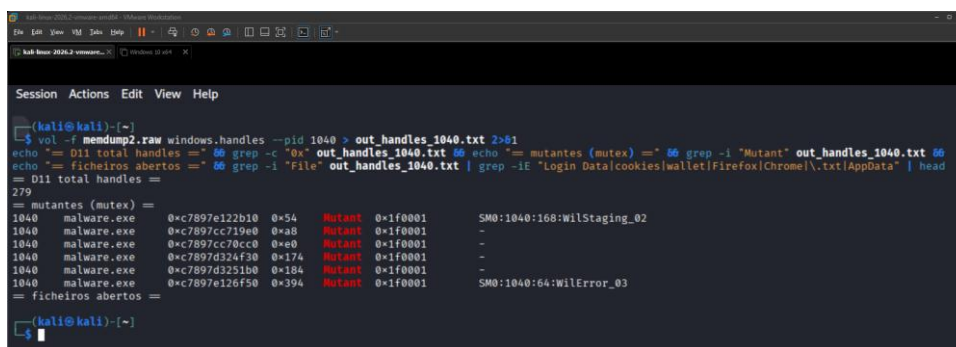
PID Process SID Name
1040 malware.exe S-1-5-21-945993646-1774408020-135794793-1001 Analista
1040 malware.exe S-1-5-21-945993646-1774408020-135794793-513 Domain Users
1040 malware.exe S-1-1-0 Everyone
1040 malware.exe S-1-5-114 Local Account (Member of Administrators)
1040 malware.exe S-1-5-32-544 Administrators
1040 malware.exe S-1-5-32-545 Users
1040 malware.exe S-1-5-4 Interactive
1040 malware.exe S-1-2-1 Console Logon (Users who are logged onto the physical console)
1040 malware.exe S-1-5-11 Authenticated Users
1040 malware.exe S-1-5-15 This Organization
1040 malware.exe S-1-5-113 Local Account
1040 malware.exe S-1-5-5-0-379692 Logon Session
1040 malware.exe S-1-2-0 Local (Users with the ability to log in locally)
1040 malware.exe S-1-5-64-10 NTLM Authentication
1040 malware.exe S-1-16-12288 High Mandatory Level
= sessão do PID 1040 =
1 - 1040 malware.exe DESKTOP-A10VAUI/Analista 2026-08-04 19:36:23.000000 UTC
(kali@kali)-[~]
└─$

```

Figura 23: windows.getsids e windows.sessions: contexto de segurança e sessão do processo suspeito.

O windows.getsids e o windows.sessions estabeleceram o contexto de segurança do processo (Figura 23). O malware.exe executa no contexto do utilizador Analista, com pertença ao grupo Administrators e nível de integridade elevado, na sessão interativa 1 do anfitrião DESKTOP-A10VAUI, com carimbo temporal 19:36:23 UTC. O nível de integridade elevado é condição do laboratório, pois detonei a amostra com privilégios de administrador, e não uma escalada de privilégios do malware, uma vez que não observei qualquer técnica de contorno do controlo de contas de utilizador.

5.4.10. Ficheiros, objetos e artefactos em memória



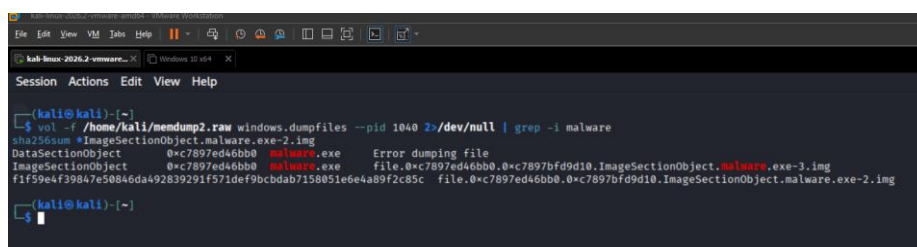
```

(kali@kali)-[~]
└─$ vol -f memdump2.raw windows.handles --pid 1040 > out_handles_1040.txt 2>&1
echo "D11 total handles =" 66 grep -c "0x" out_handles_1040.txt 66 echo " = mutantes (mutex) =" 66 grep -i "Mutant" out_handles_1040.txt 66
echo " = ficheiros abertos =" 66 grep -i "File" out_handles_1040.txt | grep -iE "Login Data|cookies|wallet|Firefox|Chrome|\.txt|AppData" | head
= D11 total handles =
279
= mutantes (mutex) =
1040 malware.exe 0xc7897e122b10 0x54 Mutant 0x1f0001 SM0:1040:168:WilStaging_02
1040 malware.exe 0xc7897cc719e0 0xa8 Mutant 0x1f0001 -
1040 malware.exe 0xc7897cc70cc0 0xe0 Mutant 0x1f0001 -
1040 malware.exe 0xc7897d324f30 0x174 Mutant 0x1f0001 -
1040 malware.exe 0xc7897d3251b0 0x184 Mutant 0x1f0001 -
1040 malware.exe 0xc7897e126f50 0x394 Mutant 0x1f0001 SM0:1040:64:WilError_03
= ficheiros abertos =
(kali@kali)-[~]
└─$

```

Figura 24: windows.handles do PID 1040: ausência de handles a repositórios de credenciais.

O windows.handles sobre o PID 1040 enumerou 279 objetos (Figura 24). Os mutexes identificados correspondem à nomenclatura genérica do runtime .NET, associada ao subsistema de relato de erros do Windows, e não constituem indicadores específicos da família. Não identifiquei, no momento da captura, handles abertos a repositórios de credenciais, resultado que interpreto na secção seguinte.



```

(kali@kali)-[~]
└─$ vol -f /home/kali/memdump2.raw windows.dumpfiles --pid 1040 2>/dev/null | grep -i malware
sha256sum *ImageSectionObject.malware.exe-2.img
DataSectionObject 0xc7897ed46bb0 malware.exe Error dumping file
ImageSectionObject 0xc7897ed46bb0 malware.exe file.0xc7897ed46bb0.0xc7897bf9d10.ImageSectionObject.malware.exe-3.img
f1f59e4f39847e50846da492839291f571def9bcbdb7158051e6e4a89f2c85c file.0xc7897ed46bb0.0xc7897bf9d10.ImageSectionObject.malware.exe-2.img
(kali@kali)-[~]
└─$

```

Figura 25: windows.dumpfiles: extração do ImageSectionObject do processo suspeito e cálculo do respetivo SHA-256.

O windows.dumpfiles extraiu o ImageSectionObject do processo suspeito (Figura 25). O binário reconstruído a partir da memória tem o valor SHA-256 f1f59e4f39847e50846da492839291f571def9bcbdb7158051e6e4a89f2c85c. Este valor difere, como

esperado, do valor do binário original em disco, por a imagem em memória ter relocações aplicadas, a tabela de importações resolvida e as secções realinhadas. Serve para comprovar a integridade do artefacto extraído, não para o comparar com o binário em disco; essa comparação faz-se pela via da atribuição, tratada em 8.12.

5.4.11. Indícios de injeção de código

```

(kali@kali)~$ vol -f /home/kali/memdump2.raw windows.malfind --pid 1040 > malfind.txt
cat malfind.txt
/home/kali/.local/share/pipx/venvs/volatility3/lib/python3.13/site-packages/volatility3/framework/deprecation.py:28: FutureWarning: This API (volatility3.plugins.windows.malware.malfind.Malfind.run) will be removed in the first release after 2026-06-07. This plugin has been renamed, please call volatility3.plugins.windows.malware.malfind.Malfind rather than volatility3.plugins.windows.malfind.Malfind.
  warnings.warn(
/home/kali/.local/share/pipx/venvs/volatility3/lib/python3.13/site-packages/volatility3/framework/deprecation.py:105: FutureWarning: This plugin (volatility3.plugins.windows.malfind.Malfind) has been renamed and will be removed in the first release after 2026-06-07. Please ensure all method calls to this plugin are replaced with calls to volatility3.plugins.windows.malware.malfind.Malfind
  warnings.warn(
Volatility 3 Framework 2.28.0

PID      Process Start VPN      End VPN Tag      Protection      CommitCharge      PrivateMemory      File output      Notes      Hexdump Disasm
-----
1040     malware.exe      0x32d0000      0x32dffff      Vads      PAGE_EXECUTE_READWRITE      1      1      Disabled      N/A
ec 70 30 4d d1 e9 00 01 ee ff ee ff 02 00 00 00 .....p0M.....
a6 00 2d 03 a4 00 2d 03 00 00 2d 03 00 00 2d 03 .....
0f 00 00 00 a8 04 2d 03 00 f0 2d 03 0e 00 00 00 .....
01 00 00 00 00 00 00 00 f0 0f 2d 03 f0 0f 2d 03 .....
          ec 70 30 4d d1 e9 00 01 ee ff ee ff 02 00 00 00 a4 00 2d 03 a4 00 2d 03 0
0 00 2d 03 00 00 2d 03 0f 00 00 00 a8 04 2d 03 00 f0 2d 03 0e 00 00 01 00 00 00 00 00 00 f0 2d 03 f0 0f 2d 03
1040     malware.exe      0x5a40000      0x5a4ffff      Vads      PAGE_EXECUTE_READWRITE      1      1      Disabled      N/A
9f ec 76 be e4 cd 00 01 ee ff ee ff 02 00 00 00 .....V.....
a4 00 a4 05 a4 00 a4 05 00 00 a4 05 00 00 a4 05 .....
0f 00 00 00 a8 04 a4 05 00 f0 a4 05 0e 00 00 00 .....
01 00 00 00 00 00 00 00 f0 0f a4 05 f0 0f a4 05 .....
          9f ec 76 be e4 cd 00 01 ee ff ee ff 02 00 00 00 a4 00 a4 05 a4 00 a4 05 0
0 00 a4 05 00 00 a4 05 0f 00 00 00 a8 04 a4 05 00 f0 a4 05 0e 00 00 01 00 00 00 00 00 00 f0 0f a4 05 f0 0f a4 05
1040     malware.exe      0x7f3b0000      0x7f3bffff      Vads      PAGE_EXECUTE_READWRITE      2      1      Disabled      N/A
ec ff ff ff 04 00 00 01 00 00 00 02 0e 03 .....h.....$.j
1c 00 00 00 68 01 04 08 18 00 00 24 27 e4 6a .....j...j..T..
00 10 e3 6a 20 d1 f4 6a cc 1f e3 6a f0 54 09 00 ...j...j..T..
00 00 00 00 40 80 3b 7f 60 80 3b 7f 80 80 3b 7f ...B.;.i...i.
          ec ff ff ff 04 00 00 01 00 00 00 02 0e 03 1c 00 00 00 68 01 04 08 1
8 00 00 24 27 e4 6a 00 10 e3 6a 20 d1 f4 6a cc 1f e3 6a f0 54 09 00 00 00 00 00 40 80 3b 7f 60 80 3b 7f 80 80 3b 7f
1040     malware.exe      0x7f3a0000      0x7f3affff      Vads      PAGE_EXECUTE_READWRITE      1      1      Disabled      N/A
00 00 00 00 97 19 00 00 00 00 00 18 00 00 00 .....
b8 00 00 00 e9 fa 09 f0 ec b8 01 00 00 00 e9 .....
f0 09 f0 ec b8 02 00 00 00 e9 e6 09 f0 ec b8 03 .....
00 00 00 e9 dc 09 f0 ec b8 04 00 00 00 e9 d2 09 .....
          00 00 00 00 97 19 00 00 00 00 00 18 00 00 00 b8 00 00 00 e9 fa 09 f
0 ec b8 01 00 00 00 e9 f0 09 f0 ec b8 02 00 00 00 e9 e6 09 f0 ec b8 03 00 00 00 e9 dc 09 f0 ec b8 04 00 00 00 e9 d2 09

```

Figura 26: windows.malfind: quatro regiões de memória com permissões de execução e escrita sem correspondência em disco.

O windows.malfind identificou, no espaço do PID 1040, quatro regiões de memória privada com permissões PAGE_EXECUTE_READWRITE (executáveis e graváveis em simultâneo), sem correspondência a ficheiro mapeado em disco, nos deslocamentos 0x32d0000, 0x5a40000, 0x7f3b0000 e 0x7f3a0000 (Figura 26). Os conteúdos hexadecimais mostram bytes executáveis, e não dados ou zeros. Páginas RWX privadas, sem suporte de ficheiro, são a assinatura clássica de injeção de código, e mapeiam para a técnica MITRE ATT&CK T1055 (Process Injection). Um binário legítimo não aloca páginas executáveis e graváveis desta natureza.

5.4.12. Atribuição de família

MalwareBazaar Database

You are currently viewing the MalwareBazaar entry for SHA256 b3d310c2853f8d633be45b6c7bf57f4d1da0ecb11a757096720621db381d0b15. While MalwareBazaar tries to identify whether the sample provided is malicious or not, there is no guarantee that a sample in MalwareBazaar is malicious.

Database Entry

Intelligence 10 | IOCs | YARA 16 | File information | Comments | Actions

SHA256 hash: b3d310c2853f8d633be45b6c7bf57f4d1da0ecb11a757096720621db381d0b15

SHA-384 hash: 40b94c41e43ac39f2d04489d5d66b25c914116c13499a9899430eb4336264107130ea99aefca0c0626ef84e19a1759

SHA1 hash: 00d10387d84851a5d0c700d766fc8b97e5ace3

MD5 hash: d05dfc1d4c5c6498694370330a3020fa

humanhash: undress-glucose-jersey-quebec

File name: b3d310c2853f8d633be45b6c7bf57f4d1da0ecb11a757096720621db381d0b15

Download: [download sample](#)

Signature: [RedLineStealer](#) [Alert](#)

File size: 307712 bytes

First seen: 2025-08-14 12:19:49 UTC

Last seen: Never

File type: exe

MIME type: application/x-dosexec

ImpHash: f345f2d4577ed5f89ec516c1f5a744 (49126 x AgentTesla, 20156 x Formbook, 12362 x SnakeKeylogger)

ssdeep: 3072:GcZqf7D34cp/0+m4yloRQYg/y81HA0PuTVakuf3R4eqCL2bBOA:GcZqf7DkmmWh81A0GTVBkNYL

Threatray: 2718 similar samples on MalwareBazaar

TLSH: T198645A58338C910DA74775D861D6719380BCA3A552E70B4FCACAB3D32740EA50A86

TrID: 69.7% (EXE) Generic CIL Executable (.NET, Mono, etc.) (73123/4/13)
10.0% (EXE) Win64 Executable (generic) (10522/11/4)
6.2% (DLL) Win32 Dynamic Link Library (generic) (6578/25/2)
4.2% (EXE) Win32 Executable (generic) (4504/4/1)
1.9% (EXE) Win16/32 Executable Delphi generic (2072/23)

Magika: pebin

dhash icon: 0000000000000000 (903 x AgentTesla, 559 x Formbook, 316 x RedLineStealer)

Reporter: Neiki

Tags: exe, RedLineStealer

Figura 27: Ficha do MalwareBazaar para a amostra: assinatura RedLineStealer, 16 detecções de fornecedores e 16 regras YARA.

A atribuição da amostra à família RedLine Stealer não se apoia num único indicador. Pelo contrário, assenta em múltiplas linhas de evidência convergentes (Figura 27). Consultando a ficha do MalwareBazaar (abuse.ch, 2025) para o binário, encontra-se a assinatura RedLineStealer. Esta classificação é fortemente suportada por 16 detecções de fornecedores e 16 regras YARA. Adicionalmente, a análise TrID da mesma ficha aponta para 69,7% de probabilidade de ser um executável .NET/CIL. Este perfil técnico coincide na perfeição com o carregamento do Common Language Runtime (CLR) que observei na memória (Figura 19). Desta forma, consigo ligar o comportamento analisado à família de forma totalmente independente.

A proveniência da amostra é íntegra e verificável. O mesmo SHA-256 confirma-se em três pontos: na fonte pública, no ambiente de análise após extração (Figura 28) e na máquina vítima (Figura 29).

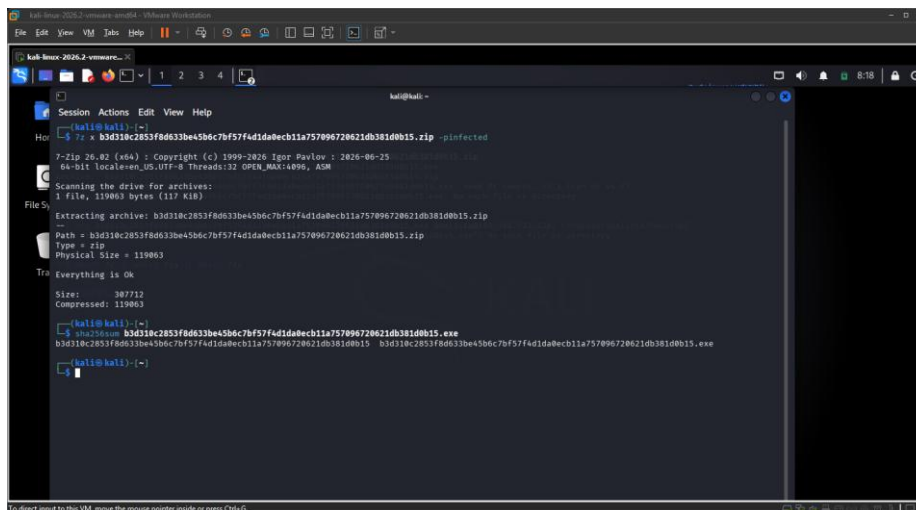


Figura 28: Extração da amostra e cálculo do SHA-256 no Kali, coincidente com o da fonte.

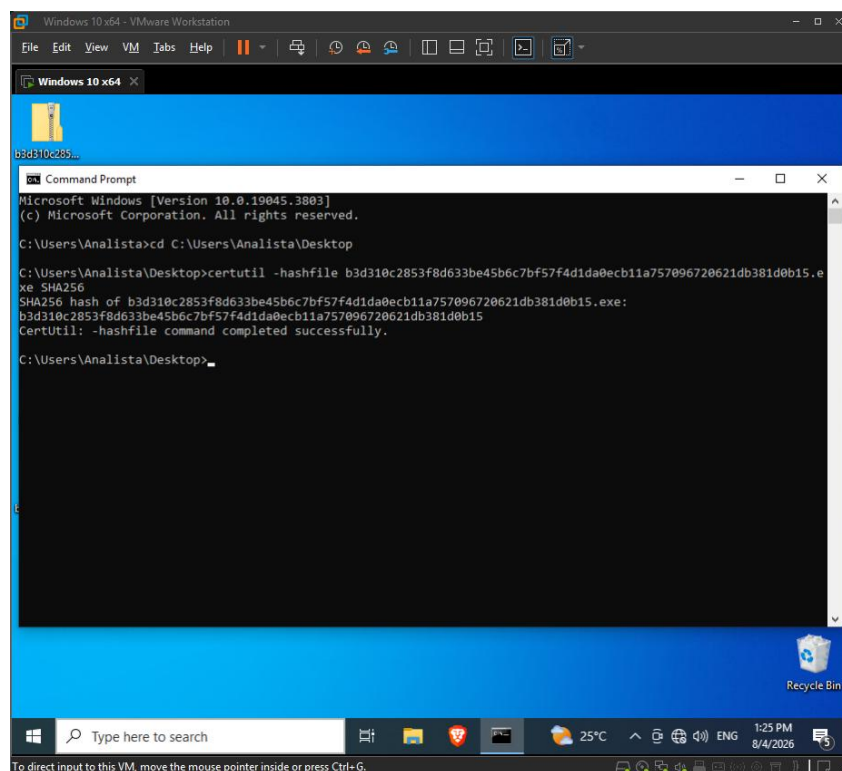


Figura 29: Cálculo do SHA-256 da amostra na máquina vítima, coincidente com os anteriores.

Ponto da cadeia	SHA-256 da amostra
Ficha MalwareBazaar	b3d310c2853f8d633be45b6c7bf57f4d1da0ecb11a757096720621db381d0b15
Ambiente de análise (Kali)	idêntico
Máquina vítima (certutil)	idêntico

De forma complementar, extraí as cadeias de texto do binário reconstruído na memória (Figura 30). Estas cadeias confirmam a capacidade de recolha da família. Nos resultados obtidos, surgem referências diretas a navegadores, tais como o chrome.exe, o msedge.exe e o opera. Encontrei também menções a carteiras de criptomoeda e a um serviço de geolocalização de IP (api.ip.sb). Adicionalmente, o código revela uma estrutura interna desenhada para enumerar os programas instalados. Todas estas cadeias demonstram que o binário tem capacidade para recolher credenciais e informação detalhada do sistema. Contudo, importa reforçar a distinção que mantenho ao longo de todo o relatório. Evidenciar a capacidade de recolha não prova, por si só, que a exfiltração dos dados tenha efetivamente acontecido.

```

(kali@kali)-[~]
$ strings file.0*c7897ed46bb0.0*c7897bfd9d10.ImageSectionObject.malware.exe.img | grep -iE "ChromeGet|AllWallets|ListOfPrograms|chromeKey|Is64Bit" | sort -u
AllWallets
ChromeGetLocalName
ChromeGetName
ChromeGetRoamingName
chromeKey
get_Is64BitOperatingSystem
ListOfPrograms
<ListOfPrograms>b__8_0

(kali@kali)-[~]
$ strings file.0*c7897ed46bb0.0*c7897bfd9d10.ImageSectionObject.malware.exe.img | grep -iE "ChromeGet|AllWallets|ListOfPrograms|chromeKey|Is64Bit" | sort -u
AllWallets
ChromeGetLocalName
ChromeGetName
ChromeGetRoamingName
chromeKey
get_Is64BitOperatingSystem
ListOfPrograms
<ListOfPrograms>b__8_0

```

Figura 30: Cadeias de texto extraídas do binário reconstruído: indicadores de capacidade de recolha e reconhecimento.

5.4.13. Resumo de indicadores de comprometimento

A tabela seguinte consolida os principais artefactos técnicos identificados durante a análise, num formato adequado à sua reutilização em plataformas de partilha de inteligência de ameaças. Cada indicador remete para o resultado que o sustenta.

Tipo de indicador	Valor	Contexto / deteção
Hash (SHA-256)	b3d310c2853f8d633be45b6c7bf57f4d1da0ecb11a757096720621db381d0b15	Amostra original em disco (certutil; ficha MalwareBazaar)
Caminho	C:\Users\Analista\Desktop\malware.exe	Local de execução (windows.cmdline)
Nome do processo	malware.exe	Binário malicioso em execução
PID	1040	Processo associado à amostra no dump
Regiões RWX	0x32d0000, 0x5a40000, 0x7f3b0000, 0x7f3a0000	Injeção de código (windows.malfind, T1055)
Assinatura interna	XHP	Nome interno do executável (Process Explorer ao vivo)
Cadeias	ChromeGetLocalName, AllWallets, api.ip.sb	Capacidade de recolha (strings do binário)

5.5. Interpretação e correlação das evidências

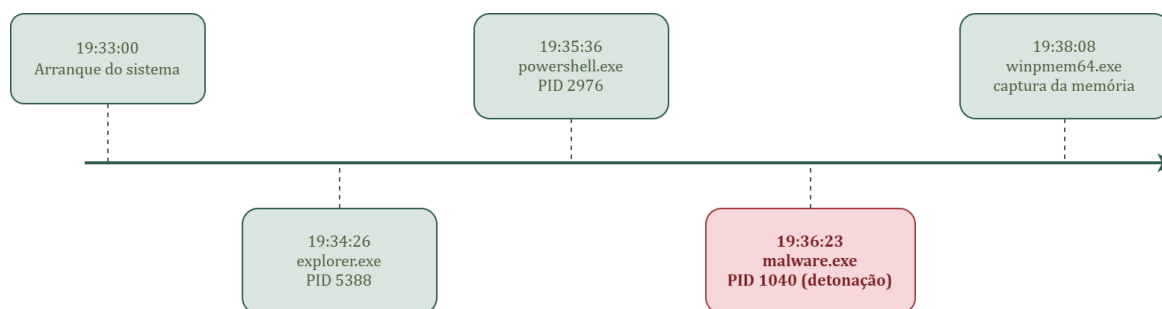
Esta secção correlaciona os achados e distingue, em cada passo, o que a imagem de memória sustenta do que decorre do enquadramento do exercício. Afirimo como facto o comportamento malicioso observado no dump; onde uma ligação é inferencial, uso a expressão *os indícios apontam para*. A identidade e a intenção do operador não são objeto desta análise, por eu próprio ter detonado a amostra de forma controlada.

5.5.1. Cronologia do incidente

Antes de correlacionar os achados isolados, reconstruo a sequência temporal do incidente a partir das horas de criação dos processos, todas obtidas do windows.pslist e expressas no fuso UTC do dump. A

convergência destas marcas temporais sustenta a narrativa e permite situar a detonação e a captura na linha do tempo do sistema (Figura 31).

Hora (UTC)	Evento	Fonte
19:33:41–51	Arranque do sistema (System, smss, services, svchost)	pslist
19:34:26	Início de explorer.exe (PID 5388), sessão do utilizador Analista	pslist
19:35:36	Início de powershell.exe (PID 2976), consola usada na detonação	pslist
19:36:23	Detonação da amostra: malware.exe (PID 1040)	pslist / sessions
19:37:49	Primeiro socket TCPv4 do PID 1040 (tentativa de comunicação)	netscan
19:38:14	Segundo socket TCPv4 do PID 1040	netscan
19:38:08–20	Aquisição da memória com o WinPmem (PID 1968)	windows.info / winpmem



Cronologia de execução (UTC) reconstruída a partir dos CreateTime dos processos

Figura 31: Cronologia de execução (UTC) reconstruída a partir das horas de criação dos processos.

A leitura da cronologia é coerente e sem lacunas. O sistema arrancou, o utilizador iniciou sessão, e a amostra foi detonada a partir da consola do PowerShell cerca de dois minutos depois. As tentativas de comunicação de rede ocorreram nos segundos seguintes à detonação, e a captura da memória foi realizada com o processo ainda ativo, pouco mais de um minuto após a execução. Este intervalo curto entre a detonação e a captura é relevante para a interpretação dos achados negativos: a janela de execução foi propositadamente breve, o que ajuda a explicar a ausência de handles a repositórios de credenciais e a não concretização da exfiltração.

5.5.2. Identificação e ausência de ocultação

A identificação do processo `malware.exe` (PID 1040), executado a partir do ambiente de trabalho com nome genérico, fixa o objeto de análise mas não é, por si só, prova de malícia. O que tem valor interpretativo é a comparação entre o `pslist` e o `psscan` (secção 8.3): a coincidência do deslocamento exclui a ocultação por desassociação da lista de processos. O processo assume-se em memória, comportamento coerente com um infostealer de recolha rápida, que privilegia a velocidade sobre a furtividade prolongada.

5.5.3. Injeção de código

As quatro regiões de memória executáveis e graváveis sem correspondência em disco (secção 8.11) são a assinatura clássica de injeção de código, que mapeia para a técnica MITRE ATT&CK T1055. Trata-se de um indicador de comportamento malicioso ativo que não depende da atribuição de família, reforçado pelo contexto `.NET` do processo (secção 8.6) e pela supressão dos metadados de compilação, que evidencia ofuscação deliberada.

5.5.4. Tentativa de comunicação não concretizada

Os dois sockets TCPv4 no estado CLOSED (secção 8.7) apontam para tentativas de contacto com infraestrutura de comando e controlo. Como estão fechados e o ambiente é isolado, o destino não é determinável e a ligação não se concretizou: afirmo a tentativa, não um canal estabelecido. O TCPView ao vivo, sem qualquer ligação externa (secção Dificuldades), cruza-se com esta leitura. O TCPView observou a execução ao vivo e o `netscan` a execução capturada no dump, o mesmo binário em execuções distintas.

5.5.5. Ausência de persistência

A ausência de serviço associado e de entrada de arranque automático (secção 8.5) é um achado com valor probatório, não uma lacuna. Confirma o perfil de recolha rápida do RedLine, que executa, recolhe e termina sem presença duradoura, pelo que este negativo reforça a atribuição em vez de a enfraquecer. A comparação com o estado limpo do sistema, documentado antes da detonação, sustenta a leitura por exclusão: nenhuma entrada de persistência nova surgiu após a execução.

5.5.6. Contexto de segurança e recolha de credenciais

O nível de integridade elevado do processo (secção 8.9) é condição do laboratório, e não uma escalada de privilégios, pois não observei qualquer contorno do controlo de contas de utilizador. A ausência de handles a repositórios de credenciais (secção 8.10) interpreta-se à luz da curta janela de execução e da ausência de conectividade: a recolha ativa não chegou a ser observada na memória. Isto delimita o que não se provou, sem infirmar a classificação como stealer, que se sustenta na injeção (9.3) e na atribuição (9.7).

5.5.7. Atribuição por convergência

A atribuição à família RedLineStealer resulta da convergência entre a extração do binário em memória (Figura 25) e a ficha pública da amostra (Figura 27), suportada por 16 deteções de fornecedores e 16 regras YARA. A natureza .NET observada em memória coincide com a caracterização técnica independente da amostra, e as cadeias internas do binário (Figura 30) confirmam a capacidade de recolha de credenciais de navegadores, carteiras e informação do sistema. A cadeia de proveniência é íntegra, com o mesmo valor de dispersão confirmado na fonte, no ambiente de análise e na vítima.

5.5.8. Leitura pelo Diamond Model

O Diamond Model de análise de intrusões (Caltagirone, Pendergast & Betz, 2013) estrutura o incidente segundo quatro vértices interligados. Ao passo que o ATT&CK responde ao *como*, o Diamond organiza o *quem*, o *o quê* e o *contra quem*, reforçando a correlação sem exigir recolha adicional: os quatro vértices ligam-se aos mesmos achados já documentados.

Vértice	Caracterização a partir da evidência
Adversário	Operador do RedLineStealer. A identidade e a intenção são contexto do laboratório, pois detonei a amostra de forma controlada; não há atribuição a um ator real.
Capacidade	RedLineStealer (.NET, 32 bits). Injeção de código confirmada (Figura 25); anti-forense por metadados suprimidos (Figura 9); capacidade de recolha de navegadores, carteiras e geolocalização de IP (Figura 29).
Infraestrutura	Canal de comando e controlo tentado, não concretizado: dois sockets fechados (Figura 19), destino não determinável, e uma referência a serviço de geolocalização de IP nas cadeias do binário.
Vítima	Anfitrião DESKTOP-A10VAUI, utilizador Analista, sessão 1 em contexto elevado (Figura 22). Alvos pretendidos: credenciais de navegadores e carteiras de criptomoeda.

Os dois eixos do modelo alinham diretamente com a classificação do incidente. O eixo da intenção, a recolha e exfiltração de credenciais e dados, corresponde à classificação primária da taxonomia (Segurança da Informação). O eixo do método, a infeção com injeção de código e tentativa de comunicação, corresponde à classificação secundária (Código Malicioso). O Diamond não substitui o ATT&CK nem a taxonomia; cruza-os, e por isso reforça a correlação sem trabalho adicional de recolha.

5.5.9. Leitura pelo Malware Behavior Catalog

O Malware Behavior Catalog, na versão 3.1 (MITRE, 2023), é um catálogo do MITRE específico para análise de malware, que complementa o ATT&CK ao cobrir objetivos que este não contempla, como a dificuldade da análise estática. Reclassifiquei os achados já obtidos segundo este vocabulário, sem recolha nova, mantendo os mesmos níveis de força.

Objetivo (MBC)	Comportamento observado	Estatuto
Execution	Injeção de código em regiões RWX privadas (Figura 25)	Confirmado
Defense Evasion	Ofuscação do binário .NET e supressão do carimbo de compilação (Figuras 12 e 27)	Confirmado
Anti-Static Analysis	Metadados de compilação suprimidos, dificultando a análise estática	Confirmado
Credential Access	Referências a navegadores e a carteiras de criptomoeda nas cadeias (Figura 29)	Capacidade
Discovery	Enumeração de programas e geolocalização de IP (Figura 29)	Capacidade
Command and Control	Sockets do processo (Figura 19)	Indício

O contributo próprio do MBC é o objetivo de dificuldade da análise estática, que o ATT&CK generaliza na técnica T1027 mas não isola. A distinção que mantenho ao longo do relatório aplica-se também aqui: os comportamentos de acesso a credenciais e de descoberta são capacidade do binário, observada nas suas cadeias internas, e não execução observada na captura. As quatro lentes utilizadas, o ATT&CK para as técnicas, o MBC para os comportamentos de malware, o Diamond Model para a estrutura da intrusão e a Taxonomia RNCISRT para a classificação do incidente, cobrem-se sem se sobreporem e sustentam o critério de correlação sem recolha adicional.

5.5.10. Quadro-síntese das correlações

O quadro seguinte resume os achados, a fonte que os sustenta, a figura de apoio, o mapeamento MITRE ATT&CK quando aplicável e o respetivo estatuto probatório (facto observado no dump ou indício inferido).

Achado	Fonte	Figura	MITRE	Estatuto
Processo suspeito PID 1040	pslist/pstree/cmdline	9,11,15	—	Facto
Ausência de ocultação	pslist vs psscan	10	—	Facto
Genealogia de execução	pstree	11	contexto	Facto
Injeção de código (RWX)	malfind	20	T1055	Facto
Tentativa de C2	netscan	14	T1071	Indício
Ausência de persistência	svcsan/autoruns	12,32	—	Facto
Contexto elevado	getsids/sessions	17	laboratório	Facto
Sem handles a credenciais	handles	18	negativo	Facto
Atribuição RedLine	dumpfiles + ficha + YARA	19,21	—	Facto
Binário .NET (CLR)	dlllist	13	—	Facto
Anti-forense (metadados)	procexp/dlllist	27,12	T1027	Indício

A leitura ATT&CK organiza-se em três níveis de confiança. A técnica T1055 (Process Injection) está confirmada por evidência direta no dump. As técnicas T1071 (comunicação de rede) e T1027 (ofuscação de ficheiros) são indícios inferidos, respetivamente, dos sockets e da supressão de metadados. Num terceiro nível, a capacidade de recolha de credenciais de navegadores (T1555.003), de descoberta de localização por IP (T1614.001) e de descoberta de software (T1518), observada nas cadeias do binário, indica o que a amostra está preparada para fazer, mas cuja execução não observei na captura. Esta separação é a disciplina de atribuição que mantenho em todo o relatório.

5.5.11. Reconstrução da narrativa do incidente

Ligando as correlações anteriores, a memória sustenta a seguinte narrativa, com grau de confiança elevado quanto ao comportamento e à família. A partir do ambiente de trabalho do utilizador Analista, o sistema executou um binário .NET de 32 bits com os metadados de compilação suprimidos, que não recorreu a ocultação na lista de processos e alojou código em quatro regiões executáveis e graváveis sem correspondência em disco, evidência de injeção. Durante a execução tentou comunicar com o exterior, sem sucesso, por o ambiente estar isolado, e não estabeleceu persistência, comportamento consistente com a lógica de recolha rápida dos infostealers. O cruzamento do binário extraído com a ficha pública atribui o artefacto à família RedLineStealer, com a natureza .NET a corroborar de forma independente, e a cadeia de custódia garante que o objeto analisado é o que foi capturado. Os detalhes desta síntese, e as respetivas limitações, são retomados nas conclusões.

6. Caso B — LockBit 4.0 (Ransomware)

Neste segundo caso replico a metodologia sobre uma amostra de natureza tática oposta: um ransomware da família LockBit 4.0. Onde o RedLine procura operar em silêncio, o LockBit atua pelo impacto direto e visível. A comparação sistemática entre as duas pegadas em memória é tratada na secção 7. Como no Caso A, separo o que fica provado a partir do dump daquilo que corresponde ao comportamento conhecido da família.

6.1. Ambiente e amostra analisada

O ambiente de laboratório, as máquinas virtuais e a instrumentação são os descritos na secção 3, revertendo-se a máquina vítima a um estado limpo antes da detonação. Assinala-se apenas o que é específico deste caso.

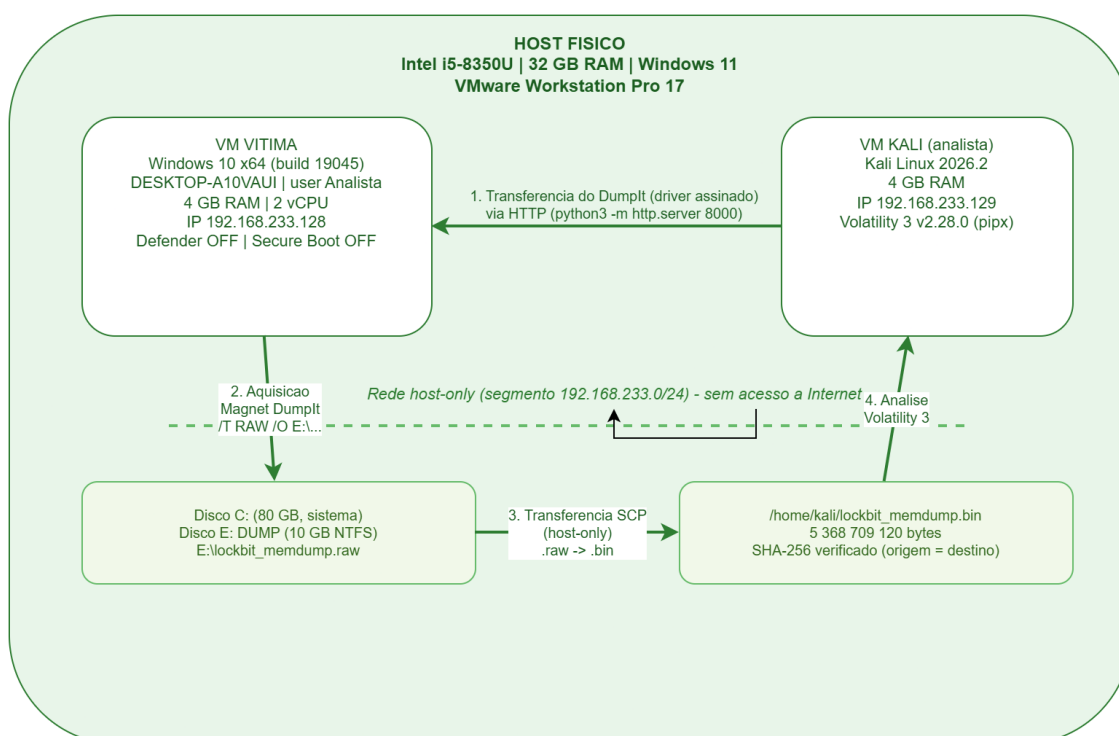


Figura 32: Arquitetura do laboratório e fluxo de aquisição do Caso B. A vítima está isolada em rede host-only; o dump é escrito no disco E: e transferido para o Kali por SCP.

Para permitir a execução do ransomware, desativei deliberadamente as proteções que o bloqueariam: a proteção em tempo real do Windows Defender, o acesso controlado a pastas, o Secure Boot e a virtualização baseada em segurança. Confirmei o isolamento de rede antes da detonação, com o adaptador em modo host-only e o tráfego para o exterior a falhar na totalidade.

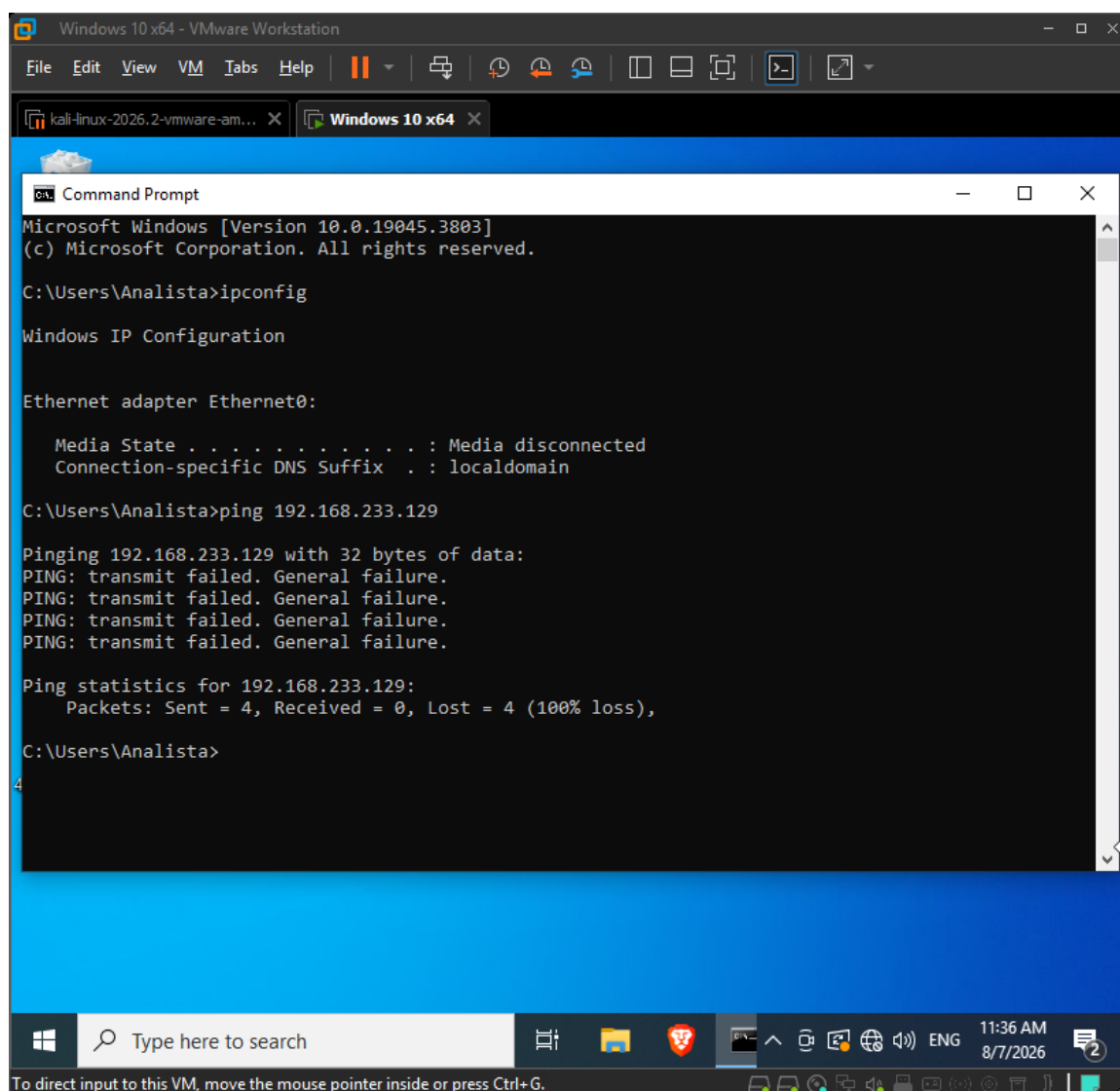


Figura 33: Confirmação do isolamento de rede na máquina vítima antes da detonação, com perda total de pacotes para um endereço externo.

A amostra provém do MalwareBazaar e corresponde à família LockBit 4.0, identificada na nota interna como LockBitGreen4.0-rc-577. Tem 61 952 bytes e o valor de dispersão SHA-256 48e2033a286775c3419bea8702a717de0b2aaf1e737ef0e6b3bf31ef6ae00eb5. Ao contrário do que fiz no Caso A, o valor de dispersão para atribuição foi retirado da ficha pública. Não efetuei um novo cálculo na máquina vítima. O motivo é simples: durante a detonação, o próprio ficheiro da amostra acabou cifrado pelo LockBit. A análise à memória viria a confirmar este facto mais tarde.

6.2. Detonação e efeitos observados no sistema vivo

Após a detonação, o ransomware cifrou de forma indiscriminada os ficheiros do sistema. Cada ficheiro passou a ostentar uma extensão hexadecimal única acrescentada ao nome original, e a varredura alcançou não só os ficheiros-isco preparados como também as ferramentas presentes no disco. A captura do sistema vivo, com o Process Explorer e o TCPView, mostra o resultado da cifra sobre os ficheiros-isco.

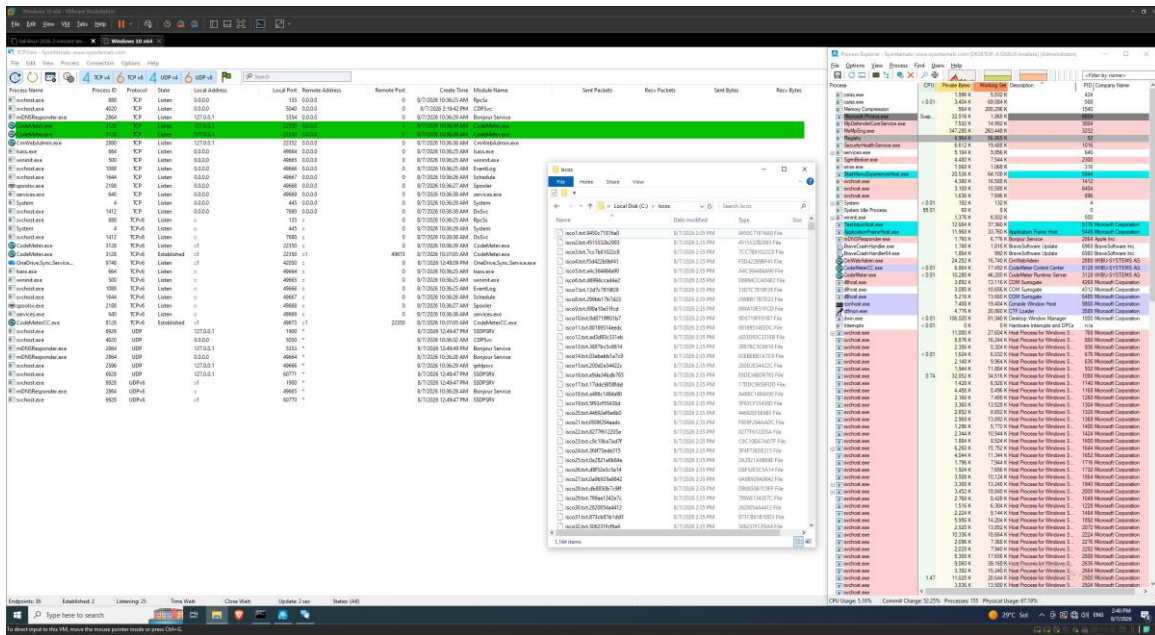


Figura 34: Estado do sistema após a detonação: os ficheiros-isco surgem cifrados com extensão hexadecimal única, observados no explorador de ficheiros e no Process Explorer.

O LockBit depositou em cada diretório uma nota de resgate com o nome Restore-My-Files.txt. O conteúdo, que li diretamente na vítima, identifica a versão do ransomware, um identificador pessoal da campanha, uma chave de acesso e a infraestrutura de dupla extorsão baseada em serviços .onion. Estes elementos viriam a revelar-se decisivos para a atribuição em memória.

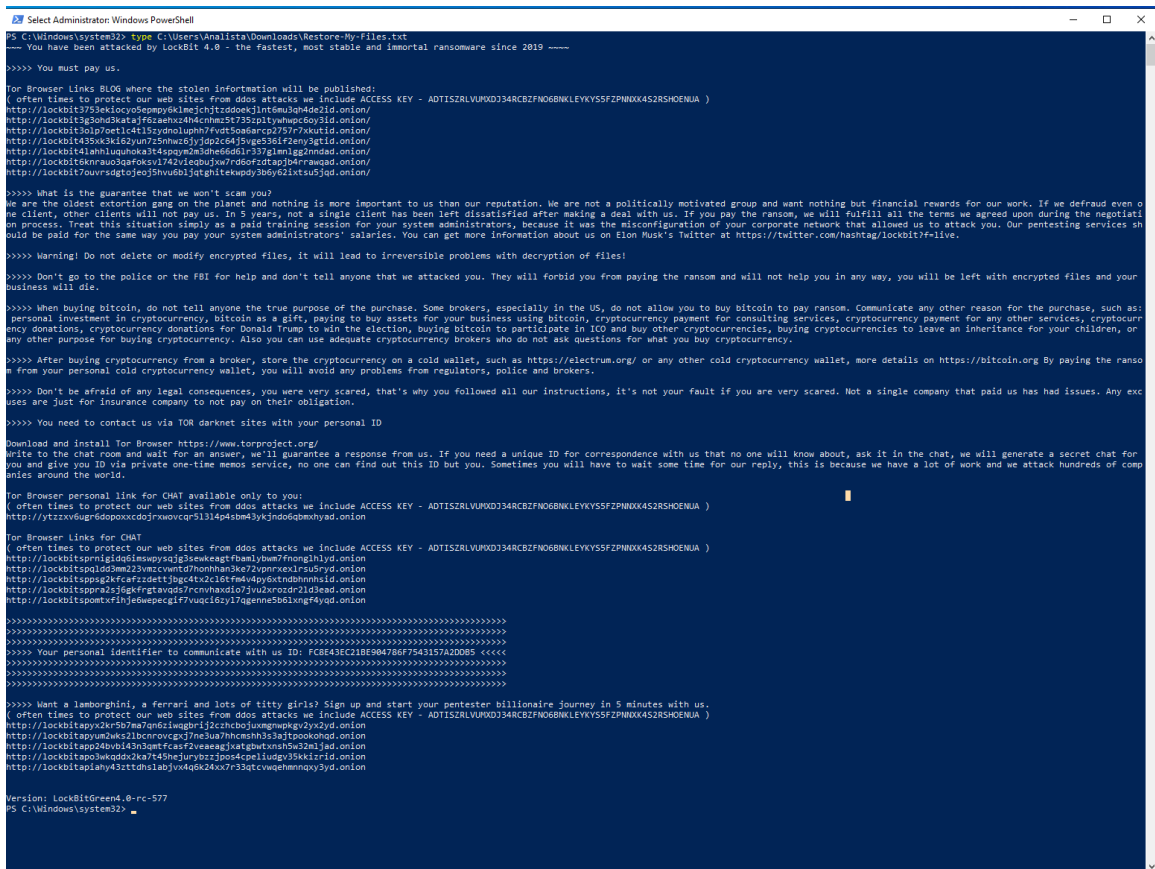
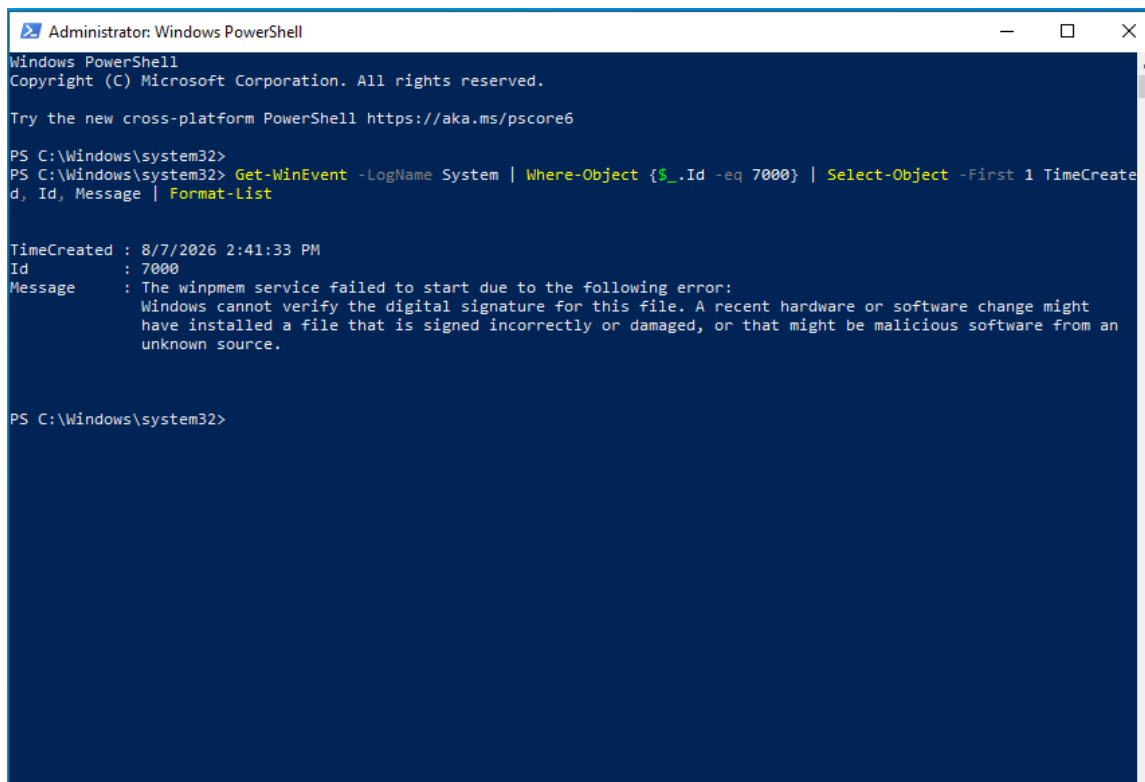


Figura 35: Nota de resgate Restore-My-Files.txt lida no sistema vivo, com a versão LockBitGreen4.0-rc-577, o identificador pessoal, a chave de acesso e os endereços .onion de extorsão.

6.3. Aquisição da memória e dificuldades

A aquisição da memória revelou-se o ponto mais exigente deste caso, e a sua resolução constitui a contribuição metodológica do trabalho. As duas ferramentas que tentei em primeiro lugar falharam pela mesma causa. O WinPmem não conseguiu iniciar o seu serviço, com o registo de eventos a indicar a impossibilidade de verificar a assinatura digital do controlador.



```
Administrator: Windows PowerShell
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Try the new cross-platform PowerShell https://aka.ms/pscore6

PS C:\Windows\system32>
PS C:\Windows\system32> Get-WinEvent -LogName System | Where-Object {$_.Id -eq 7000} | Select-Object -First 1 TimeCreated, Id, Message | Format-List

TimeCreated : 8/7/2026 2:41:33 PM
Id           : 7000
Message      : The winpmem service failed to start due to the following error:
               Windows cannot verify the digital signature for this file. A recent hardware or software change might
               have installed a file that is signed incorrectly or damaged, or that might be malicious software from an
               unknown source.

PS C:\Windows\system32>
```

Figura 36: Falha do WinPmem na iniciação do serviço, por recusa da assinatura digital do controlador de kernel (Driver Signature Enforcement).

A causa reside no mecanismo de imposição de assinatura de controladores do Windows, o Driver Signature Enforcement. Este sistema recusa qualquer controlador de kernel que não esteja assinado segundo o programa WHQL. Antes de chegar a esta conclusão, confirmei que o Secure Boot, a virtualização baseada em segurança e o Defender estavam todos desativados. Logo, nenhum deles estava a bloquear o processo. A recusa provinha, pura e exclusivamente, da falta de assinatura digital. O FTK Imager falhou pelo mesmo motivo. Para agravar o cenário, a própria imagem ISO do FTK acabou por ser cifrada pelo ransomware.

```

Administrator: Windows PowerShell
PS C:\Windows\system32> dir C:\Ferramentas\

Directory: C:\Ferramentas

Mode                LastWriteTime         Length Name
----                -
d-----            8/7/2026   2:35 PM                SysinternalsSuite
-a----            8/7/2026   4:02 PM         630600 DumpIt.exe
-a----            8/7/2026   2:35 PM    408186972 FTK Imager 8.2.0.iso.7e0a77162c5e
-a----            8/7/2026   2:35 PM          6227 Restore-My-Files.txt
-a----            8/4/2026   9:48 AM         764416 winpmem64.exe

PS C:\Windows\system32>

```

Figura 37: A imagem ISO do FTK Imager surge cifrada pelo LockBit, com a extensão hexadecimal característica acrescentada ao nome.

Para contornar esta limitação, recorri ao Magnet DumpIt. Como o seu controlador é assinado, o mecanismo de imposição de assinatura aceitou a execução sem problemas. A transferência do executável, do Kali para a máquina vítima, foi feita através de um servidor HTTP simples a correr sobre a rede host-only. Esta abordagem garantiu duas coisas essenciais: não expor a máquina anfitriã e manter a vítima sem qualquer acesso à Internet. A aquisição ocorreu com sucesso. O processo gerou a imagem lockbit_memdump.bin, com os exatos 5 368 709 120 bytes esperados.

```

Destination path:      \??\E:\lockbit_memdump.bin
Computer name:         DESKTOP-A10VAUI

--> Proceed with the acquisition ? [y/n] y

[+] Information:
Dump Type:             Raw Memory Dump

[+] Machine Information:
Windows version:       10.0.19045
MachineId:             946F4D56-7957-8CFD-2402-B86DFA5766C1
TimeStamp:             134305886723954596
Cr3:                   0x1ad002
KdCopyDataBlock:       0xffffffff8001fb10c08
KdDebuggerData:        0xffffffff8002020b20
KdpDataBlockEncoded:   0xffffffff80020250b40

Current date/time:     [2026-08-07 (YYYY-MM-DD) 15:04:32 (UTC)]
+ Processing... Done.

Acquisition finished at: [2026-08-07 (YYYY-MM-DD) 15:05:12 (UTC)]
Time elapsed:          0:40 minutes:seconds (40 secs)

Created file size:      5368709120 bytes (5120 Mb)
Total physical memory size: 4095 Mb

NtStatus (troubleshooting): 0x00000000
Total of written pages:  1048333
Total of inaccessible pages: 0
Total of accessible pages: 1048333

SHA-256: C2F8D20D120A33556AD75E11F352530AEFF76686338763706EED801AD4380FB3

JSON path:             E:\lockbit_memdump.json

PS C:\Windows\system32>

```

Figura 38: Conclusão da aquisição com o Magnet DumpIt e cálculo do valor de dispersão SHA-256 na origem.

Registo, como facto relevante para a interpretação posterior, que o processo do ransomware já tinha terminado no momento da captura. O LockBit segue um padrão de cifra-e-sai: executa a cifra e encerra-se,

sem estabelecer persistência. A imagem de memória contém, por isso, os artefactos residuais do impacto e não o processo em execução, o que condiciona a análise que se segue.

6.4. Integridade da evidência e cadeia de custódia

Apliquei o procedimento de integridade e cadeia de custódia definido na secção 4: cálculo do SHA-256 na origem e recálculo no destino após a transferência por SCP. Os dois valores coincidem, o que confirma que a imagem não foi alterada no transporte.

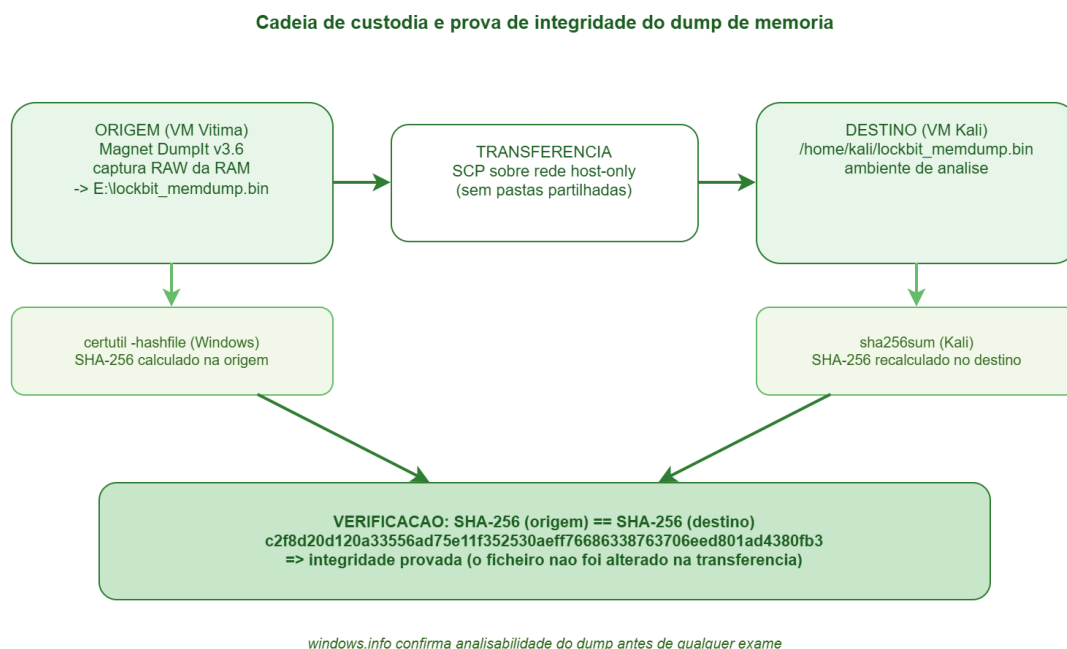


Figura 39: Cadeia de custódia e prova de integridade do dump, com o valor de dispersão coincidente entre a origem e o destino.

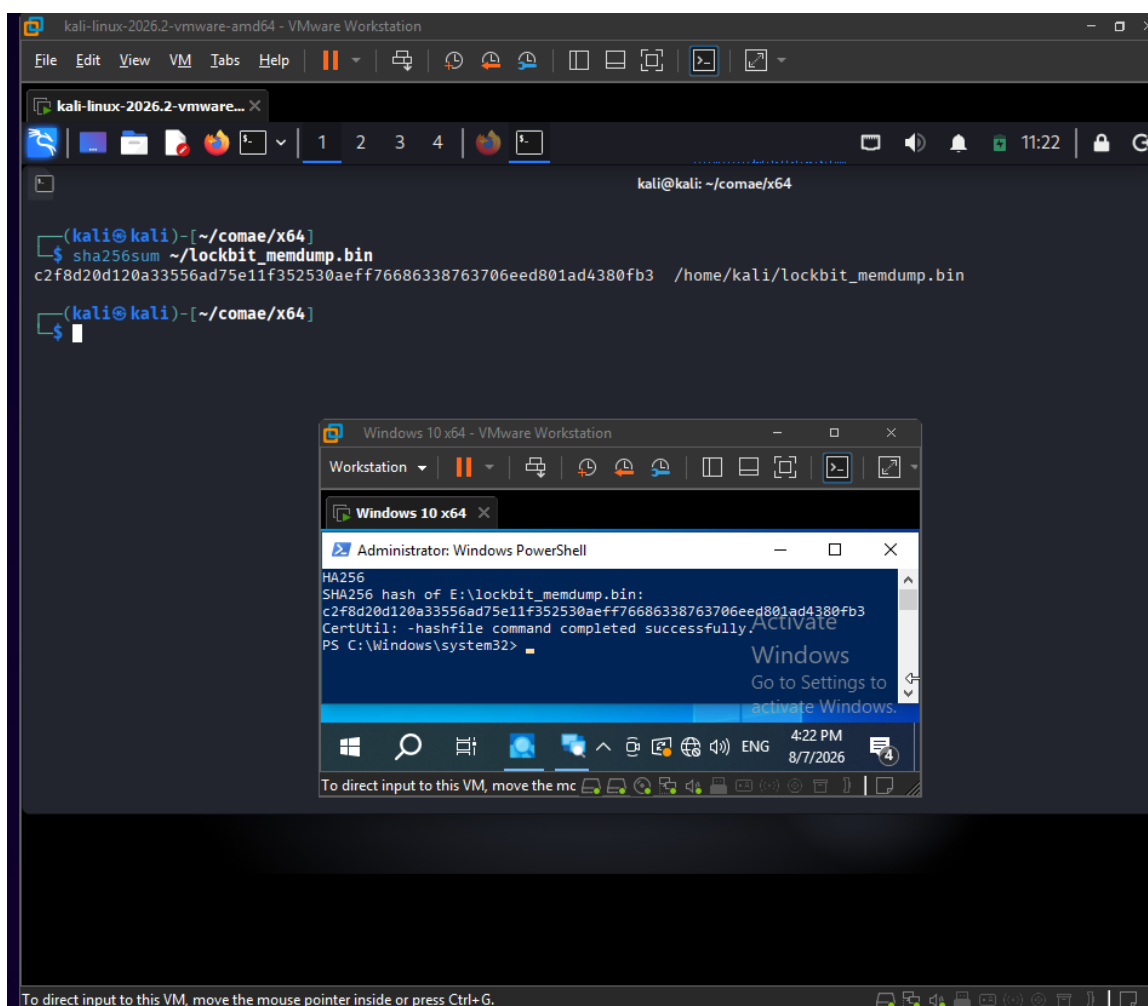


Figura 40: Verificação do SHA-256 na origem (certutil) e no destino (sha256sum), com valores coincidentes.

O valor de dispersão verificado é `c2f8d20d120a33556ad75e11f352530aeff76686338763706eed801ad4380fb3`. Confirmada a integridade, o passo seguinte foi validar a viabilidade de análise da imagem com o plugin `windows.info`, que fez casar os símbolos do kernel e devolveu um perfil coerente do sistema operativo, com a arquitetura de 64 bits e uma marca temporal do sistema alinhada com o momento da captura.

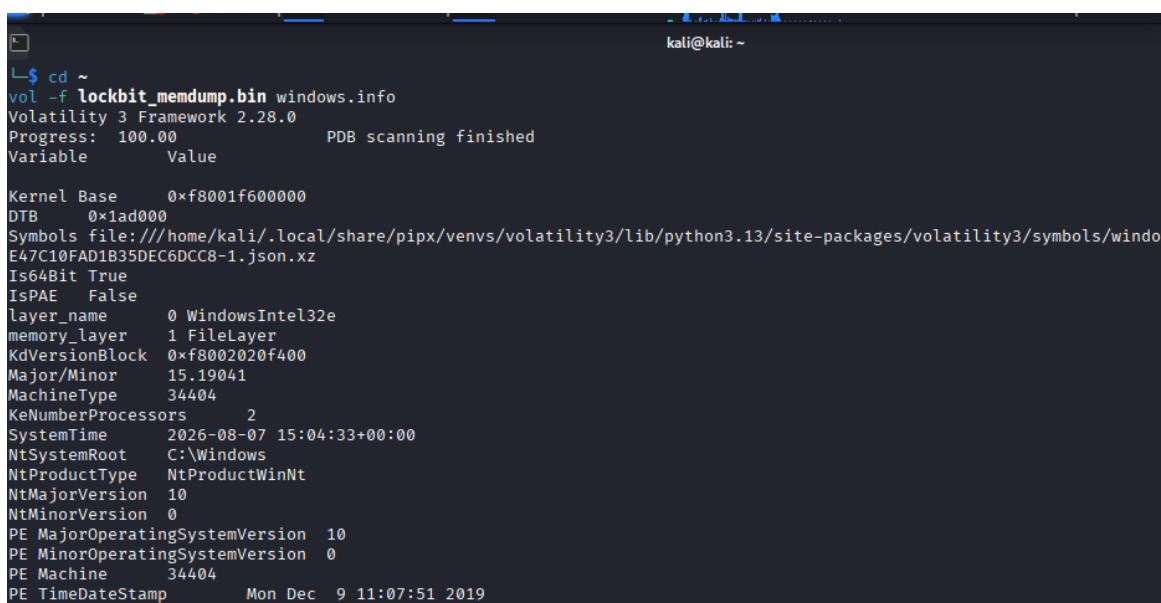


Figura 41: windows.info confirma a viabilidade de análise da imagem e devolve o perfil do sistema operativo coerente com a captura.

6.5. Resultados da análise em memória

Apresento os resultados pela ordem lógica da investigação. A leitura que os estrutura, e que distingue este caso do Caso A, é a seguinte: por o processo ter terminado antes da captura, os artefactos de execução ativa estão ausentes da memória, ao passo que os artefactos de impacto persistente estão presentes. Esta assimetria não constitui uma limitação da análise; constitui um achado sobre a natureza do adversário e sobre o momento da aquisição.

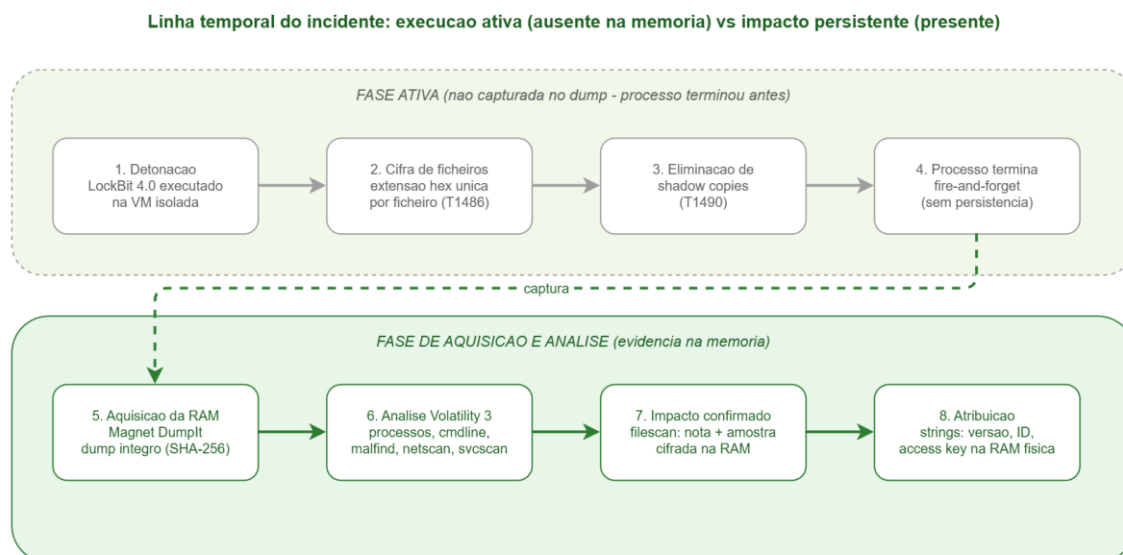


Figura 42: Linha temporal do incidente. A fase ativa (cifra, eliminação de cópias de sombra, término do processo) não foi capturada na memória; a fase de impacto persistente foi recuperada da imagem.

Processos. O plugin windows.psscan, que varre as estruturas de memória e recupera processos já desassociados da lista ativa, enumerou os processos residuais sem revelar qualquer processo atribuível ao ransomware, por nome ou por identificador da amostra. A cadeia recuperada mostra o explorador a lançar uma sessão de PowerShell interativa e processos de linha de comandos já terminados, além do próprio DumpIt usado na aquisição. A ausência do processo do LockBit confirma, a partir do dump, o comportamento de cifra-e-sai.

Comandos e consolas. O windows.cmdline mostra a linha de comando do DumpIt, com a escrita da imagem para o disco E:, o que documenta a própria aquisição. Não recuperou qualquer comando de destruição, como o vssadmin, o wmic ou o bcdedit, e o windows.consoles não recuperou histórico de comandos maliciosos. A eliminação das cópias de sombra, que confirmei no sistema vivo, não ficou registada como comando na memória.

```

kali@kali: ~
(kali@kali)-[~]
$ grep -iE "vssadmin|wmic|bcdedit|wbadmin|shadow|delete|lockbit|.exe\" out_cmdline.txt
7424 DumpIt.exe "C:\Ferramentas\DumpIt.exe" /T RAW /O E:\lockbit_memdump.raw

(kali@kali)-[~]
$ grep -E "(4224|4016|10068|9068|7772)" out_cmdline.txt
4224 explorer.exe C:\Windows\Explorer.EXE
10068 powershell.exe "C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe"
9068 conhost.exe \??\C:\Windows\system32\conhost.exe 0x4

(kali@kali)-[~]
$

(kali@kali)-[~]
$ vol -f lockbit_memdump.bin windows.consoles > out_consoles.txt
cat out_consoles.txt
Volatility 3 Framework 2.28.0 PDB scanning finished

PID Process ConsoleInfo Property Address Data
9068 conhost.exe 0x7ff698182e68 _CONSOLE_INFORMATION N/A Console Information Not Found

(kali@kali)-[~]
$

(kali@kali)-[~]
$

```

Figura 43: windows.cmdline e windows.consoles: a linha de comando do DumpIt está presente; não há comandos de destruição nem histórico de consola atribuível ao ransomware.

A eliminação das cópias de sombra foi observada diretamente no sistema vivo, onde a enumeração de cópias de sombra não devolveu qualquer item.

```

Administrator: Windows PowerShell

vssadmin 1.1 - Volume Shadow Copy Service administrative command-line tool
C) Copyright 2001-2013 Microsoft Corp.

o items found that satisfy the query.
S C:\Windows\system32>

```

Figura 44: No sistema vivo, a enumeração de cópias de sombra (`vssadmin list shadows`) não devolve itens, indício da técnica de inibição da recuperação (T1490).

Para manter o rigor da análise, distingo dois planos. O efeito da técnica de inibição da recuperação do sistema está provado: as cópias de sombra foram eliminadas. O mecanismo, ou seja, o comando concreto que as eliminou, não foi capturado na memória; a sua atribuição ao LockBit assenta no comportamento documentado da família. Os indícios apontam para a execução do `vssadmin` pelo ransomware, coerente com a família, mas a evidência em memória não o prova de forma autónoma.

Injeção de código. O `windows.malfind` identificou regiões de memória com permissões de execução e escrita, mas todas pertencem a processos legítimos: o motor do Windows Defender, o SearchApp, o SmartScreen e o motor .NET do PowerShell. Nenhuma região apresenta cabeçalho de executável em memória privada nem se associa a um processo suspeito. Não há, portanto, injeção de código atribuível ao ransomware. A ausência acompanha o término do processo antes da captura, e contrasta com o Caso A, onde o `malfind` revelou quatro regiões de injeção no processo do RedLine ainda em execução.

Estas regiões com permissões de execução e escrita não indiciam injeção maliciosa, mas um comportamento legítimo decorrente da compilação em tempo de execução (Just-In-Time). Estes motores traduzem código intermédio para código nativo durante a execução, o que obriga o sistema a alocar páginas simultaneamente graváveis, para alojar o código compilado, e executáveis, para o correr. O `windows.malfind` deteta esta alocação, tratando-se de um falso positivo validado pela natureza documentada destes processos, ao contrário da injeção anómala verificada no Caso A.

PID	Process	Start	End	Tag	Protection	CommitCharge	PrivateMemory	File	output	Notes	Hexdump	Disasm
3232	MsMpEng.exe	0x1d3a4c00000	0x1d3a4c00fff	VadS	PAGE_EXECUTE_READWRITE	1	1	1	Disabled	Function prologue		
3232	MsMpEng.exe	0x1d3a4b10000	0x1d3a4b10fff	VadS	PAGE_EXECUTE_READWRITE	1	1	1	Disabled	Function prologue		
3232	MsMpEng.exe	0x1d3a4b20000	0x1d3a4b20fff	VadS	PAGE_EXECUTE_READWRITE	1	1	1	Disabled	Function prologue		
3232	MsMpEng.exe	0x1d3a4b60000	0x1d3a4b61fff	VadS	PAGE_EXECUTE_READWRITE	2	1	1	Disabled	Function prologue		
3232	MsMpEng.exe	0x1d3a4bf0000	0x1d3a4bf2fff	VadS	PAGE_EXECUTE_READWRITE	3	1	1	Disabled	Function prologue		
3232	MsMpEng.exe	0x1d3a4da0000	0x1d3a4da0fff	VadS	PAGE_EXECUTE_READWRITE	1	1	1	Disabled	Function prologue		
3232	MsMpEng.exe	0x1d3a4d00000	0x1d3a4d01fff	VadS	PAGE_EXECUTE_READWRITE	2	1	1	Disabled	Function prologue		
3232	MsMpEng.exe	0x1d3a4de0000	0x1d3a4deefff	VadS	PAGE_EXECUTE_READWRITE	15	1	1	Disabled	Function prologue		
3232	MsMpEng.exe	0x1d3a6710000	0x1d3a681cfff	VadS	PAGE_EXECUTE_READWRITE	269	1	1	Disabled	N/A		
3232	MsMpEng.exe	0x1d3a6600000	0x1d3a670cfff	VadS	PAGE_EXECUTE_READWRITE	269	1	1	Disabled	N/A		
3232	MsMpEng.exe	0x1d3ad8e0000	0x1d3ad9ecfff	VadS	PAGE_EXECUTE_READWRITE	269	1	1	Disabled	N/A		
3232	MsMpEng.exe	0x1d3ada00000	0x1d3adb0cfff	VadS	PAGE_EXECUTE_READWRITE	269	1	1	Disabled	N/A		
3232	MsMpEng.exe	0x1d3b55f0000	0x1d3b55f0fff	VadS	PAGE_EXECUTE_READWRITE	1	1	1	Disabled	Function prologue		
5240	SearchApp.exe	0x28797740000	0x2879775ffff	VadS	PAGE_EXECUTE_READWRITE	6	1	1	Disabled	N/A		
5240	SearchApp.exe	0x288a8900000	0x288a8963fff	VadS	PAGE_EXECUTE_READWRITE	5	1	1	Disabled	N/A		
10700	RuntimeBroker.exe	0x25638e70000	0x25638e70fff	Vad	PAGE_EXECUTE_READWRITE	0	0	0	Disabled	N/A		
10068	powershell.exe	0x1e7472b0000	0x1e7472bffff	VadS	PAGE_EXECUTE_READWRITE	9	1	1	Disabled	N/A		
10068	powershell.exe	0x1e747240000	0x1e74724cfff	VadS	PAGE_EXECUTE_READWRITE	1	1	1	Disabled	N/A		
10068	powershell.exe	0x7df4cf2f0000	0x7df4cf2ffff	VadS	PAGE_EXECUTE_READWRITE	1	1	1	Disabled	N/A		
10068	powershell.exe	0x7df4cf300000	0x7df4cf39fff	VadS	PAGE_EXECUTE_READWRITE	2	1	1	Disabled	N/A		
4592	smartscreen.exe	0x1a08b590000	0x1a08b5affff	VadS	PAGE_EXECUTE_READWRITE	1	1	1	Disabled	N/A		

Figura 45: windows.malfind: as regiões com permissões de execução e escrita pertencem a processos legítimos (Defender, SearchApp, SmartScreen, PowerShell), sem injeção atribuível ao ransomware.

Ficheiros em memória. O windows.filescan recuperou os objetos de ficheiro residentes na imagem. A pesquisa dirigida devolveu dois artefactos decisivos: a nota de resgate Restore-My-Files.txt e o próprio ficheiro da amostra, no ambiente de trabalho, cifrado e com a extensão hexadecimal .c9478dabb503 acrescentada. Este segundo achado prova, a partir da memória, que a cifra alcançou o próprio dropper, e liga o valor de dispersão de atribuição ao artefacto cifrado.

Path	Name	Size
0xd70bb76865d0	\Users\Analista\Desktop\48e2033a286775c3419bea8702a717de0b2aaf1e737ef0e6b3bf31ef6ae00eb5.zip.c9478dabb503	
0xd70bb76a00c0	\Users\Analista\AppData\Roaming\Microsoft\Internet Explorer\Quick Launch\User Pinned\TaskBar\Restore-My-Files.txt	

Figura 46: windows.filescan: a nota de resgate e o ficheiro da amostra cifrado (extensão .c9478dabb503) estão referenciados na memória, prova direta do impacto de cifra.

Rede. O windows.netscan não revelou qualquer ligação atribuível ao ransomware. Os únicos artefactos de rede correspondem a serviços legítimos: consultas de resolução de nomes do svchost, o licenciamento CodeMeter em interface de retorno e o OneDrive. Não há ligações estabelecidas para o exterior nem tráfego para a infraestrutura .onion. A ausência de comunicação de comando e controlo é coerente com o isolamento deliberado da vítima e com o término do processo. A dupla extorsão anunciada na nota não se concretizou em rede, por decisão do desenho experimental.

IP	Port	Protocol	State	PID	Process	Time
0xd70bb5082720	192.168.233.128	TCPv4		53		2026-08-07 15:00:25.000000 UTC
0xd70bb5152a20	:::1	TCPv6	22350	49673	ESTABLISHED	3128 CodeMeter.exe 2026-08-07 09:37:05.000000 UTC
0xd70bb5153a20	:::1	TCPv6	49673	22350	ESTABLISHED	8128 CodeMeterCC.ex 2026-08-07 09:37:05.000000 UTC
0xd70bb6de61b0	:::1	TCPv6	42050	0	LISTENING	9748 OneDrive.Sync. 2026-08-07 11:48:09.000000 UTC
0xd70bb9c25a20	192.168.233.128	TCPv4	60528	53	CLOSED	1048 svchost.exe 2026-08-07 15:00:25.000000 UTC
0xd70bba1dd4a0	192.168.233.128	TCPv4	60607	53	CLOSED	1048 svchost.exe 2026-08-07 15:00:25.000000 UTC

Figura 47: windows.netscan: os artefactos de rede correspondem a serviços legítimos; não há ligação atribuível ao ransomware nem tráfego para a infraestrutura de extorsão.

Serviços e persistência. O windows.svcscan enumerou 1541 serviços, nenhum atribuível ao ransomware. Os únicos serviços cuja designação contém terminologia de cifra são componentes nativos do Windows, como o BitLocker e o sistema de ficheiros cifrados. A pesquisa por serviços com nome do LockBit ou com binários no perfil do utilizador não devolveu resultados. A ausência de serviço malicioso corrobora, a

partir da memória, a ausência de persistência já observada nas chaves de arranque do sistema vivo, e acompanha o comportamento de cifra-e-sai.

```
(kali@kali)-[~]
$ grep -iE "encrypt|restore|lockbit" out_svcscan.txt | grep -iE "RUNNING|STOPPED" | head -8
grep -icE "lockbit|\\\\Users\\\\\\\\Analista" out_svcscan.txt
wc -l out_svcscan.txt
0x2a2084477e0 164 N/A SERVICE_BOOT_START SERVICE_RUNNING SERVICE_KERNEL_DRIVER fvevol BitLocker
r Drive Encryption Filter Driver \Driver\fvevol System32\DRIVERS\fvevol.sys -
0x2a20843e3e0 141 N/A SERVICE_DEMAND_START SERVICE_STOPPED SERVICE_WIN32_SHARE_PROCESS EFS
ncrypting File System (EFS) N/A %SystemRoot%\System32\lsass.exe %SystemRoot%\system32\efssvc.dll
0x2a208438350 49 N/A SERVICE_DEMAND_START SERVICE_STOPPED SERVICE_WIN32_SHARE_PROCESS BDESVC
itLocker Drive Encryption Service N/A %SystemRoot%\System32\svchost.exe -k netsvcs -p %SystemRoot%\Sys
tem32\bdesvc.dll
0x2a208490db0 603 N/A SERVICE_DEMAND_START SERVICE_STOPPED SERVICE_WIN32_SHARE_PROCESS WEPHOSTSVC
VC Windows Encryption Provider Host Service N/A %systemroot%\system32\svchost.exe -k WepHostSvc
roup %systemroot%\system32\wepostsvc.dll
0x2a2084477e0 164 N/A SERVICE_BOOT_START SERVICE_RUNNING SERVICE_KERNEL_DRIVER fvevol BitLocker
r Drive Encryption Filter Driver \Driver\fvevol System32\DRIVERS\fvevol.sys -
0x2a208438350 49 N/A SERVICE_DEMAND_START SERVICE_STOPPED SERVICE_WIN32_SHARE_PROCESS BDESVC
itLocker Drive Encryption Service N/A %SystemRoot%\System32\svchost.exe -k netsvcs -p %SystemRoot%\Sys
tem32\bdesvc.dll
0x2a20843e3e0 141 N/A SERVICE_DEMAND_START SERVICE_STOPPED SERVICE_WIN32_SHARE_PROCESS EFS
ncrypting File System (EFS) N/A %SystemRoot%\System32\lsass.exe %SystemRoot%\system32\efssvc.dll
0x2a208490db0 603 N/A SERVICE_DEMAND_START SERVICE_STOPPED SERVICE_WIN32_SHARE_PROCESS WEPHOSTSVC
VC Windows Encryption Provider Host Service N/A %systemroot%\system32\svchost.exe -k WepHostSvc
roup %systemroot%\system32\wepostsvc.dll
0
1541 out_svcscan.txt
(kali@kali)-[~]
$
```

Figura 48: windows.svcscan: de 1541 serviços, nenhum é atribuível ao ransomware; a pesquisa dirigida não devolve ocorrências do LockBit.

```
Administrator: Windows PowerShell
PS C:\Windows\system32> Get-ItemProperty "HKLM:\Software\Microsoft\Windows\CurrentVersion\Run"

SecurityHealth           : C:\Windows\system32\SecurityHealthSystray.exe
VMware User Process      : "C:\Program Files\VMware\VMware Tools\vmtoolsd.exe" -n vmusr
PSPath                   : Microsoft.PowerShell.Core\Registry::HKEY_LOCAL_MACHINE\Software\Microsoft\Windows\CurrentVersion\
                           Run
PSParentPath              : Microsoft.PowerShell.Core\Registry::HKEY_LOCAL_MACHINE\Software\Microsoft\Windows\CurrentVersion
PSChildName              : Run
PSDrive                   : HKLM
PSProvider                 : Microsoft.PowerShell.Core\Registry

PS C:\Windows\system32> Get-ItemProperty "HKCU:\Software\Microsoft\Windows\CurrentVersion\Run"

OneDrive                  : "C:\Users\Analista\AppData\Local\Microsoft\OneDrive\OneDrive
                           .exe" /background
MicrosoftEdgeAutoLaunch_621DF56883A40F46AE7A2F794432AA5 : "C:\Program Files
                           (x86)\Microsoft\Edge\Application\msedge.exe"
                           --no-startup-window --win-session-start
PSPath                   : Microsoft.PowerShell.Core\Registry::HKEY_CURRENT_USER\Softwa
                           re\Microsoft\Windows\CurrentVersion\Run
PSParentPath              : Microsoft.PowerShell.Core\Registry::HKEY_CURRENT_USER\Softwa
                           re\Microsoft\Windows\CurrentVersion
PSChildName              : Run
PSDrive                   : HKCU
PSProvider                 : Microsoft.PowerShell.Core\Registry

PS C:\Windows\system32>
```

Figura 49: No sistema vivo, as chaves de arranque automático contêm apenas entradas legítimas; o LockBit não estabeleceu persistência.

Objetos de sincronização. O windows.mutantscan enumerou 977 objetos de exclusão mútua. Nenhum é atribuível ao ransomware; os únicos com padrão hexadecimal pertencem a caches do SmartScreen. O identificador da campanha presente na nota não se encontra referenciado como objeto de exclusão mútua na memória. A ausência do objeto de sincronização, que impediria a dupla execução do LockBit, condiz com o término do processo, que liberta o objeto.

```

kali@kali: ~
(kali@kali)-[~]
$ vol -f lockbit_memdump.bin windows.mutantscan > out_mutantscan.txt
wc -l out_mutantscan.txt
977 out_mutantscan.txt          PDB scanning finished

(kali@kali)-[~]
$ grep -iE "lockbit|[0-9A-F]{32}|[0-9a-f]{32}" out_mutantscan.txt
0xd70bb9792210 SmartScreenRemoteDatacustomSettings2439d40683e4e07b5d55bda78d5f2d0f
0xd70bb97928d0 SmartScreenRemoteDataedgeSettings2439d40683e4e07b5d55bda78d5f2d0f
0xd70bb9793590 SmartScreenRemoteDatacustomSynchronousLookupUri2439d40683e4e07b5d55bda78d5f2d0f
0xd70bb9793650 SmartScreenLocalDataadownloadCachee5c954ecadf010d6f17be64958b05ad00
0xd70bb9793710 SmartScreenRemoteDataawarnStateCachee5c954ecadf010d6f17be64958b05ad00
0xd70bb9793d10 SmartScreenRemoteDataatopTraffice2439d40683e4e07b5d55bda78d5f2d0f
0xd70bb9794250 SmartScreenLocalDataauriCachee5c954ecadf010d6f17be64958b05ad00
0xd70bba673290 SmartScreenLocalDataadownloadCachee5d8fb3a69da4541ca8dafc6b27a34c2a
0xd70bba6734d0 SmartScreenRemoteDataawarnStateCachee5d8fb3a69da4541ca8dafc6b27a34c2a
0xd70bba673a10 SmartScreenRemoteDatacustomSettings63108f3ffcdc7e5be432814fd619355b
0xd70bba673b90 SmartScreenRemoteDatacustomSynchronousLookupUri63108f3ffcdc7e5be432814fd619355b
0xd70bba674490 SmartScreenLocalDataauriCachee5d8fb3a69da4541ca8dafc6b27a34c2a
0xd70bba674910 SmartScreenRemoteDataatopTraffice63108f3ffcdc7e5be432814fd619355b
0xd70bba6749d0 SmartScreenRemoteDataedgeSettings63108f3ffcdc7e5be432814fd619355b

(kali@kali)-[~]
$
(kali@kali)-[~]
$ grep -icE "lockbit|FC8E43EC" out_mutantscan.txt
wc -l out_mutantscan.txt
0
977 out_mutantscan.txt

(kali@kali)-[~]
$

```

Figura 50: windows.mutantscan: os objetos de exclusão mútua com padrão hexadecimal pertencem ao SmartScreen; a pesquisa pelo identificador da campanha não devolve ocorrências.

6.6. Atribuição da família em memória

A versão 2.28.0 do Volatility 3 não disponibiliza o plugin de varredura YARA sobre regiões de processo, pelo que recorri a uma abordagem de varredura de cadeias de texto sobre a imagem física. Esta escolha é tecnicamente mais adequada ao caso, precisamente por o alvo já não possuir regiões de processo ativas: a varredura da imagem física alcança páginas residuais que uma varredura por processo não cobriria. Usei como âncoras os identificadores da campanha extraídos da nota de resgate.

A varredura recuperou da memória física a versão do ransomware, o identificador pessoal, a chave de acesso e a própria nota de resgate, todos coincidentes com o que observei no sistema vivo. Recuperou ainda o comando que leu a nota, o que situa temporalmente a interação. A presença destes identificadores na memória física confirma a atribuição à campanha LockBit a partir da evidência principal, e não apenas por fonte externa, o que a torna mais robusta do que a atribuição do Caso A, apoiada sobretudo na ficha pública.

```

kali@kali: ~
(kali@kali)-[~]
$ printf '%s\n' "LockBitGreen4.0-rc-577" "Restore-My-Files" "FC8E43EC21BE904786F7543157A2DDB5" "ADTISZRLVUMXDJ34RCBZFNO6BNKLEYKYS5FZPNNXK4S2RSHOENUA" > patterns.txt
strings -a lockbit_memdump.bin | grep -iaf patterns.txt | sort -u | head -40
-a-----
8/7/2026      2:35 PM          6227 Restore-My-Files.txt
( often times to protect our web sites from ddos attacks we include ACCESS KEY - ADTISZRLVUMXDJ34RCBZFNO6BNKLEYK
YS5FZPNNXK4S2RSHOENUA )
RESTORE-MY-FILES.t
Restore-My-Files.txt
type C:\Users\Analista\Downloads\Restore-My-Files.txt
Version: LockBitGreen4.0-rc-577
>>>> Your personal identifier to communicate with us ID: FC8E43EC21BE904786F7543157A2DDB5 <<<<<

(kali@kali)-[~]
$
(kali@kali)-[~]
$

```

Figura 51: Varredura de cadeias sobre a imagem física: recuperação da versão LockBitGreen4.0-rc-577, do identificador pessoal e da chave de acesso, coincidentes com a nota de resgate.

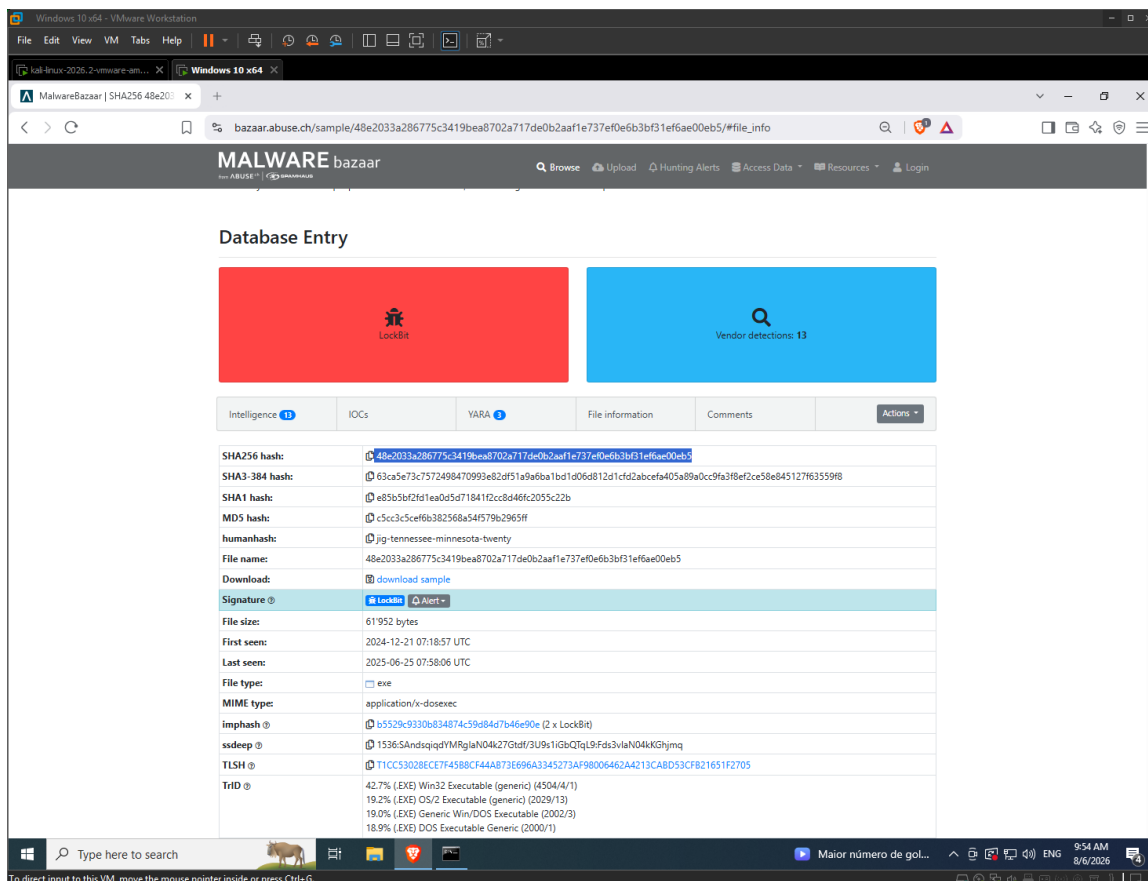


Figura 52: Ficha do MalwareBazaar para a amostra, com a assinatura LockBit, as detecções de fornecedores e as regras YARA de terceiros que sustentam a atribuição.

6.7. Quadro de indicadores de comprometimento

O quadro seguinte consolida os artefactos técnicos identificados, distinguindo o que foi provado a partir do dump do que é atribuído ao comportamento da família.

Achado	Fonte	MITRE / Estatuto
Cifra de ficheiros com extensão hexadecimal única	Sistema vivo; filescan (nota e amostra cifrada); strings	T1486 — Provado
Nota de resgate Restore-My-Files.txt em cada diretório	Sistema vivo; filescan; strings	T1486 — Provado
Eliminação das cópias de sombra (efeito)	Sistema vivo (vssadmin list shadows sem itens)	T1490 — Efeito provado
Comando de eliminação das cópias de sombra (mecanismo)	Não capturado em memória	T1490 — Atribuído à família
Injeção de código	malfind (apenas falsos positivos legítimos)	T1055 — Não sustentado
Comunicação de comando e controlo / exfiltração	netscan (sem ligação atribuível)	Não observado (isolamento)
Ausência de persistência	svcsan; chaves de arranque	Coerente com cifra-e-sai
Identificadores da campanha em memória física	strings (versão, identificador, chave)	Atribuição provada do dump

7. Análise comparativa: Furtividade contra Destruição

A captura das duas amostras no mesmo ambiente de laboratório permite comparar, lado a lado, o rasto que cada família deixa na memória volátil. Ambas são ameaças graves, mas a forma como interagem com o sistema operativo obriga o analista a orientar a investigação para plugins diferentes do Volatility. A comparação não se resume a uma oposição entre furtividade e ruído: mostra uma diferença de fundo no

momento em que cada amostra foi capturada face ao seu ciclo de vida, e no que esse momento permite recuperar.

7.1. Eixo central: o momento da aquisição

A distinção mais consequente entre os dois casos não está na família do malware, mas na relação entre o instante da aquisição e o ciclo de vida de cada amostra. O RedLine foi capturado em execução: o seu processo estava vivo na memória, o que tornou recuperáveis os artefactos de execução ativa, como a injeção de código, os sockets de rede, as bibliotecas carregadas e os identificadores de segurança do processo. O LockBit foi capturado após o facto: o processo já se tinha encerrado, pelo que a memória preserva o impacto persistente, como a nota de resgate e os ficheiros cifrados, mas não os artefactos de execução ativa.

Esta assimetria tem uma consequência prática para o método. Perante o RedLine, basta ler a presença dos artefactos: o que se procura está na imagem. Perante o LockBit, o analista tem de interpretar também a ausência. A inexistência de processo, de injeção, de objeto de sincronização e de ligação de rede não resulta de uma falha da análise, mas do comportamento de um adversário que cifra e encerra. Interpretar a ausência é o que separa a enumeração de artefactos da análise forense.

7.2. Quadro comparativo de artefactos em memória

O quadro cruza o comportamento das duas famílias com os indicadores recolhidos na memória volátil e com o plugin do Volatility que os revela em cada caso.

Tática / Indicador	RedLine Stealer (Infostealer)	LockBit (Ransomware)
Objetivo principal	Recolha de credenciais e exfiltração silenciosa	Cifra em massa e indisponibilização (extorsão)
Estado no momento da captura	Processo vivo (PID 1040)	Processo já terminado (cifra-e-sai)
Plugin revelador	malfind e netscan	filescan e strings sobre a imagem física
Injeção de código (T1055)	Quatro regiões de execução e escrita no processo	Ausente; malfind só com falsos positivos legítimos
Rasto em ficheiros	Mínimo; leitura de dados de navegadores	Massivo; cifra dos ficheiros e da própria amostra
Recuperação do sistema	Não aplicável	Eliminação das cópias de sombra (T1490, efeito)
Comunicação de rede	Dois sockets residuais (indício de C2)	Ausente; sem ligação atribuível (isolamento)
Atribuição em memória	Sobretudo por ficha pública e cadeias	Identificadores da campanha na imagem física
Classificação RNCSIRT primária	Segurança da Informação (exfiltração)	Disponibilidade (indisponibilização)

7.3. Conclusão do estudo comparativo

O estudo demonstra que não existe um guião único para a análise forense de memória. A estratégia do perito tem de adaptar-se à natureza tática do adversário e, sobretudo, ao momento em que a imagem é capturada. Perante a suspeita de um infostealer, a análise deve dirigir-se à procura de injeções anómalas e de ligações de rede estabelecidas nos primeiros instantes de execução, enquanto o processo permanece vivo. Perante um ransomware, a memória deve ser lida pelo impacto persistente, com a identificação da nota de resgate, dos ficheiros cifrados e dos identificadores da campanha, e com a leitura da ausência de execução ativa como assinatura do comportamento de cifra-e-sai.

Ambas as amostras foram dissecadas com sucesso, cada uma pelo método adequado à sua natureza e ao seu estado de captura. A comparação sustenta a tese de que a informação útil extraída da memória depende tanto do que se recupera como da correta interpretação daquilo que, legitimamente, não está lá. Mantive ao longo dos dois casos a mesma cautela probatória, separando o que os dados provam do que a literatura atribui, sem a qual nenhuma conclusão forense se sustenta.

A principal limitação deste estudo é a assimetria da visibilidade no momento da captura: o RedLine foi analisado com o processo ativo, o LockBit já depois de terminado. Esta diferença condiciona o que cada imagem permite recuperar e deve ser tida em conta ao comparar os dois casos. Ainda assim, reforça a conclusão central, a de que as ferramentas forenses de memória, por si só, não bastam: exigem sempre a interpretação humana para ler tanto as presenças como as ausências que a memória preserva.

8. Classificação na Taxonomia RNCSIRT

A classificação do incidente segue a Taxonomia Comum da Rede Nacional de CSIRT, na versão 3.3 (RNCSIRT, 2023). Esta taxonomia estabelece uma regra de prioridade: quando são aplicáveis várias classificações, a classificação primária corresponde ao objetivo do agente, e a secundária ao meio empregue para o concretizar. Apliquei esta regra ao comportamento observado, e não a uma cópia de trabalhos anteriores.

8.1. Classificação do Caso A (RedLine)

Nível	Classe	Tipo de incidente	Justificação
Primária	Segurança da Informação	Exfiltração de Informação	O objetivo do RedLine é exfiltrar credenciais e dados do host para o exterior. Evento correlacionado: comprometimento de credenciais ou dados pessoais.
Secundária	Código Malicioso	Sistema Infetado	O meio é o host infetado com malware ativo em memória, com injeção de código confirmada por malfind.

A classificação primária como Exfiltração de Informação assenta na intenção do agente, tal como a regra da taxonomia determina, e independentemente de a exfiltração se ter ou não concretizado no ambiente isolado. Na descrição do incidente deixo claro, por honestidade probatória, que a exfiltração foi tentada e não concretizada: os sockets estão fechados, não se identificaram handles a credenciais e o ambiente é isolado. A taxonomia classifica pela intenção; a interpretação forense distingue intenção de concretização. Ambas coexistem sem contradição.

Não utilizei a classe `Intrusões`, por não haver acesso interativo manual: o acesso ao sistema é obtido pela execução do próprio malware, comportamento coberto pela classe `Código Malicioso`. A ausência de persistência estabelecida, por sua vez, é uma característica do comportamento observado, e não uma classe autónoma.

8.2. Classificação do Caso B (LockBit)

Classifico o incidente segundo a Taxonomia Comum da Rede Nacional de CSIRT, na versão 3.3, o referencial adotado em Portugal pelo CERT.PT. A taxonomia estabelece que, quando são aplicáveis várias classificações, a primária corresponde ao objetivo do agente e a secundária ao meio empregue para o concretizar. Apliquei esta regra ao comportamento observado, e não por analogia com o Caso A, cuja classificação é deliberadamente distinta.

Nível	Classe	Tipo de incidente	Justificação
Primária	Disponibilidade	Indisponibilização por ransomware	O objetivo materializado é impedir o acesso aos dados pela cifra em massa, provado pela cifra dos ficheiros e da própria amostra e pela eliminação das cópias de sombra.
Secundária	Código Malicioso	Sistema infetado	O meio é a execução da amostra LockBit na vítima, atribuída pela recuperação dos identificadores da campanha em memória.

A classificação primária como Disponibilidade assenta no objetivo do agente, a indisponibilização dos dados pela cifra, distinto do objetivo de exfiltração que classificou o Caso A. A nota de resgate anuncia dupla extorsão, com publicação de dados em infraestrutura .onion, o que sugeriria também um incidente de Segurança da Informação. Para ser exato na análise, não elevo esta dimensão a classe primária: não se identificou exfiltração, os sockets para o exterior estão ausentes e o ambiente é isolado por desenho. A ameaça de divulgação permanece como comportamento atribuído à família, não como facto provado. Não utilizei a classe Intrusões, por não haver acesso interativo manual; o acesso ao sistema é obtido pela execução do próprio malware, comportamento coberto pela classe Código Malicioso.

9. Limitações e conclusões gerais

A principal limitação decorre da natureza da fonte. Um dump de memória ao vivo não é um instantâneo perfeito do estado do sistema num único momento. Como a aquisição percorre a memória física de forma sequencial enquanto o sistema continua a executar, páginas lidas no início e no fim do processo correspondem a instantes ligeiramente diferentes. Desta não-atomicidade pode resultar a inconsistência parcial de algumas estruturas em memória, fenómeno conhecido como *memory smear*. É esta, e não o risco de perda por não capturar a tempo, a limitação real do ambiente controlado: capturei num momento por mim escolhido, com o processo ativo, pelo que a perda por volatilidade não se coloca; o que se coloca é a possibilidade de o retrato da memória não ser inteiramente coerente por ter sido colhido de um alvo em movimento.

A ausência de uma baseline em memória do próprio sistema é uma segunda limitação. Não disponho de um dump anterior à detonação, pelo que a distinção entre atividade legítima do Windows e atividade anómala assenta no conhecimento dos processos normais do sistema e na literatura, e não numa comparação direta com o estado não contaminado da mesma máquina. Mitiguei parcialmente esta limitação com a captura, ao vivo, do estado limpo do sistema pelos utilitários Sysinternals antes da detonação, o que sustenta a atribuição por exclusão de qualquer processo, ligação ou persistência que surgisse depois.

A análise incide exclusivamente sobre a memória. Artefactos que residem noutras camadas, como o ficheiro de hibernação, o ficheiro de paginação ou os artefactos persistentes em disco, ficam fora do âmbito definido para este trabalho. A caracterização que apresento é a que a memória permite sustentar, e não uma reconstrução a partir de todas as fontes que um sistema Windows disponibilizaria.

O ambiente é, além disso, simulado. A amostra foi detonada numa máquina virtual isolada, em rede host-only, com o Windows Defender desligado para permitir a execução, condições que não reproduzem um sistema de produção. O valor da análise reside na demonstração do método de aquisição e análise de memória e na correlação das evidências, e não na representatividade do ambiente face a um incidente real. A ausência de infraestrutura de comando e controlo alcançável explica, de resto, que a exfiltração não se tenha concretizado.

9.1. Conclusões gerais

O estudo cumpriu o objetivo de analisar forensemente a memória volátil de dois sistemas comprometidos e de comparar a pegada deixada por duas ameaças de natureza oposta. No Caso A, a análise sustenta a presença e a execução de uma amostra da família RedLineStealer, com injeção de código, recolha de credenciais e uma tentativa de comunicação de rede não concretizada, sem persistência estabelecida. No Caso B, a análise confirma o impacto de uma amostra LockBit 4.0, com cifra de ficheiros, nota de resgate e eliminação das cópias de sombra, sem que o processo malicioso sobrevivesse à captura.

Do ponto de vista do método, o trabalho demonstra a aquisição de memória com registo de integridade e cadeia de custódia verificável, e a análise com o Volatility 3 por linha de comandos, corroborada pelos utilitários Sysinternals. A dificuldade encontrada na aquisição, a impossibilidade de analisar a imagem obtida em modo PTE Remapping, converteu-se numa contribuição metodológica concreta: a recomendação de forçar o modo PhysicalMemory quando o alvo do exame é o Volatility 3.

A comparação entre os dois casos sustenta a conclusão central do estudo: a pegada de uma ameaça em memória depende tanto da sua natureza tática como do instante da aquisição face ao ciclo de vida do processo. O RedLine, capturado em execução, revelou-se pela injeção de código e pelos artefactos de rede; o LockBit, capturado após o término, revelou-se pelo impacto persistente da cifra e pela ausência dos artefactos de execução ativa. A investigação forense de memória exige, assim, adaptar a estratégia ao momento do incidente e interpretar tanto as presenças como as ausências que a imagem preserva.

O relatório reflete os factos apurados a partir da imagem de memória analisada, até à data da sua emissão.

Referências

- Caltagirone, S., Pendergast, A., & Betz, C. (2013). The Diamond Model of Intrusion Analysis. Center for Cyber Intelligence Analysis and Threat Research. <https://www.activeresponse.org/wp-content/uploads/2013/07/diamond.pdf>
- International Organization for Standardization. (2012). ISO/IEC 27037:2012 — Information technology, security techniques: Guidelines for identification, collection, acquisition and preservation of digital evidence. Genebra: ISO/IEC. <https://www.iso.org/standard/44381.html>
- MITRE. (2023). Malware Behavior Catalog (MBC) v3.1. The MITRE Corporation. <https://github.com/MBCProject/mbc-markdown>
- MITRE ATT&CK. (2025). Enterprise Matrix. The MITRE Corporation. <https://attack.mitre.org/>
- National Institute of Standards and Technology. (2006). Guide to integrating forensic techniques into incident response (NIST Special Publication 800-86). Gaithersburg, MD: NIST. <https://csrc.nist.gov/pubs/sp/800/86/final>
- Rede Nacional de CSIRT. (2023). Taxonomia comum da Rede Nacional de CSIRT (v3.3). Lisboa: Centro Nacional de Cibersegurança. https://www.redecsirt.pt/files/RNCSIRT_Taxonomia_v3.3.pdf
- Sebenta AFD, Ivo Rosa (2025). Análise forense digital [Material didático não publicado]. Curso Técnico Superior Profissional em Cibersegurança, ISTECS — Instituto Superior de Tecnologias Avançadas.
- Tkachenko, E. (2022). RedLine Stealer malware detection. SOC Prime. <https://socprime.com/blog/redline-stealer-malware-detection/>
- Volatility Foundation. (2025). The Volatility Framework, versão 3. <https://github.com/volatilityfoundation/volatility3>
- abuse.ch. (2025). MalwareBazaar — Malware sample database. <https://bazaar.abuse.ch/>

Identificação

David Miranda — n.º